

Master's Thesis

Tabular Prior-Data Fitted Networks: Architectural Insights and Novel Approaches to Posterior Predictive Modeling

Department of Statistics
Ludwig-Maximilians-Universität München

Alexander Ulbrich

Munich, April 23th, 2026



Submitted in partial fulfillment of the requirements for the degree of M. Sc.
Supervised by Prof. Thomas Nagler

Abstract

Prior-Data Fitted Networks (PFNs) shift much of the statistical burden of supervised learning from dataset-specific optimization to large-scale pretraining. In the tabular setting, this idea is instantiated by TabPFN, which performs prediction by conditioning on a context dataset in a single forward pass. This thesis investigates how architectural design and pretraining-prior choice affect the performance of TabPFN-style models, and additionally explores new approaches to posterior predictive modeling in regression.

To this end, several prototype models were implemented and evaluated. The study compares different pretraining priors, including a simple linear prior and a more complex SCM-based prior, and analyzes architectural differences between TabPFN-V1 and TabPFN-V2. In the regression setting, two extensions of the original discretized posterior predictive distribution are introduced: a spline-based variant and a Flow Matching-based generative variant.

The empirical results show that strong PFN performance depends critically on the match between prior complexity and model capacity. A pretraining prior that is too simple leads to worse downstream performance, while increasing model complexity alone does not compensate for this mismatch. At the same time, competitive TabPFN-style prototypes can already be obtained with substantially fewer computational resources and simpler priors than in the original TabPFN training setup. In regression, the spline-based approach emerges as a particularly practical refinement of the existing framework, whereas the Flow Matching approach appears promising but requires further investigation.

Contents

1	Introduction	1
2	Preliminaries	2
2.1	Transformer Networks	2
2.2	Prior-Data Fitted Networks (PFNs)	3
2.2.1	Prior-fitting on synthetic datasets	5
2.2.2	Training objective	5
2.3	TabPFN	6
2.3.1	TabPFN-V1	6
2.3.2	TabPFN-V2	6
2.3.3	TabPFN-V2.5	10
2.4	Flow Matching	11
3	Related Work	14
4	PPD Modelling for Regression	16
4.1	How TabPFN models the PPD	16
4.2	PPD as a B-spline	16
4.3	PPD-Modeling via Flow Matching	19
5	Experiments	22
5.1	Model Implementation	22
5.2	Pretraining Priors	24
5.2.1	Simple-prior	25
5.2.2	SCM-prior	26
5.3	Synthetic classification experiments	28
5.3.1	Classification on the simple-prior	29
5.3.2	Classification on the SCM-prior	31
5.3.3	Synthetic performance and generalization	33
5.4	Benchmarking	36
5.4.1	Binary classification benchmarking	36
5.4.2	Regression benchmarking	42
6	Discussion and Limitations	46
7	Conclusion	50
A	Appendix	5
A.1	Explorative analysis of the latent space	5
A.2	Overview of TabPFN Versions	6
A.3	Additional details on the sweep for the simple-prior	7
A.4	Additional details on the sweep for the SCM prior	13
A.5	Performance over training at the dataset level	17
A.6	Additional details on regression benchmarking	20
A.7	Reproducibility and replicability	20

B Electronic appendix

21

Abbreviations

PFN	Prior-Data Fitted Network
PPD	Posterior Predictive Distribution
FFN	Feed-Forward Network
MLP	Multi-Layer Perceptron
NLL	Negative Log-Likelihood
RMSE	Root Mean Squared Error

Mathematical symbols

$\mathbf{x}$	Input / feature vector
y	Scalar target variable
$\mathcal{D}$	Observed supervised dataset
D	Context dataset available to the model
θ	Trainable model parameters
φ	Latent model parameters
$p_\theta(y \mid \mathbf{x}, D)$	Learned PPD approximation
$p(\varphi)$	Prior distribution over latent model parameters
$\pi_{\theta,j}(\mathbf{x}, D)$	Mixture weight of basis function j
$\mathcal{B}_j(y)$	j -th normalized B-spline basis function
$N_{r,d}(y)$	B-spline basis function with index r and degree d
τ_r	Knot of the spline basis
J	Number of buckets
M	Number of spline basis functions
p	Degree of the spline basis
$\ell(\theta)$	Training objective / loss function
$\mathbf{1}(\cdot)$	Indicator function
μ	Mean
σ	Standard deviation

Model and training hyperparameters

k	Embedding dimension of the transformer
n_{heads}	Number of attention heads
d_k	Embedding dimension per transformer head
m	Hidden dimension of the feed-forward / MLP blocks
n_{layers}	Number of transformer layers
n_{datasets}	Number of datasets seen during training
n_{max}	Maximum amount of samples in one dataset (pretraining)
d_{max}	Maximum amount of features in one dataset (pretraining)
batch_size	Number of datasets per training batch
accum_steps	Number of gradient accumulation steps
η_{base}	Base learning rate
η_{min}	Minimum learning rate during cosine annealing

1 Introduction

Tabular machine learning has long been an important area of research, as a substantial share of real-world data is stored in structured tabular form (Borisov et al., 2022). Compared with fields such as natural language processing, however, tabular data has historically received considerably less attention (see Van Breugel and Van Der Schaar, 2024). At the same time, tabular datasets exhibit characteristics that make them particularly challenging: they are highly heterogeneous and strongly shaped by the domain in which they are collected, whether in finance, medicine, or the social sciences. As Hollmann et al. (2025) note, even identical values can carry entirely different meanings across datasets depending on context. Classical machine learning methods, however, do not transfer knowledge across datasets; instead, they are typically trained separately and specifically for each individual task.

Despite their heterogeneity, tabular datasets often exhibit recurring structural patterns, which is reflected in the strong cross-dataset generalization of tabular foundation models such as TabPFN (see Hollmann et al., 2023, 2025). The core idea of TabPFN is to pretrain on a large number of synthetic datasets, enabling the model to capture not merely dataset-specific regularities but a more general learning procedure. In this sense, the model learns the learning algorithm itself. At inference time, TabPFN performs prediction through in-context learning, a mechanism also central to modern large language models, allowing it to make predictions in a single forward pass without parameter updates. Owing to its extensive pretraining, TabPFN is particularly strong in low-data regimes (see Hollmann et al., 2023, 2025).

This line of work has attracted substantial attention in recent years. Newer versions of TabPFN have been released with modified architectures (see Hollmann et al., 2023, 2025, Grinsztajn et al., 2025), alternative approaches have been proposed that vary the pretraining data (see Qu et al., 2025, 2026), and architectural variants have been introduced to improve computational efficiency (see Zeng et al., 2025). There are even lightweight educational implementations such as NanoTabPFN (see Pfefferle et al., 2025), which demonstrate that a minimal working prototype can already achieve competitive performance. What remains comparatively sparse, however, are controlled and systematic investigations of which specific components of the TabPFN framework matter most, such as hyperparameter choices, attention mechanisms, or alternative ways of modeling predictions.

Against this background, the present thesis aims to make the following main contributions:

- a controlled empirical analysis of how hyperparameter choices affect the convergence behavior of TabPFN prototypes,
- an investigation of how the complexity of the pretraining prior influences downstream model performance,
- and the introduction of two novel methods for applying TabPFN for regression.

2 Preliminaries

This chapter introduces the theoretical foundations. We consider a supervised learning problem with feature vectors $\mathbf{X} \in \mathcal{X} \subseteq \mathbb{R}^d$ and targets $Y \in \mathcal{Y}$. We observe an i.i.d. training sample

$$\mathcal{D}_n = \{(Y_i, \mathbf{X}_i)\}_{i=1}^n \quad (1)$$

drawn from an unknown distribution p_0 over $\mathcal{X} \times \mathcal{Y}$. In the binary classification case, $\mathcal{Y} = \{0, 1\}$, whereas in the regression case, $\mathcal{Y} \subseteq \mathbb{R}$.

2.1 Transformer Networks

Transformers were originally introduced for sequence transduction (see Vaswani et al., 2017), but their core mechanism—self-attention—has since proven useful in many other domains (see Khan et al., 2022, Islam et al., 2024). We present the standard transformer encoder block in a form that remains consistent with later sections of this thesis, following the notation of Thickstun (2021). Concretely, we view the input as a collection of n observations (e.g., words, image patches, or rows of a tabular dataset), each initially represented by a vector in $\mathbb{R}^d$. These inputs are first mapped into a shared embedding space of dimension k via a learned encoder. We denote the resulting token representations by

$$\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_n)^\top \in \mathbb{R}^{n \times k}. \quad (2)$$

A transformer block is then a parametrized function that maps $\mathbf{X}$ to an output of the same shape,

$$f_\theta : \mathbb{R}^{n \times k} \rightarrow \mathbb{R}^{n \times k}, \quad (3)$$

and multiple such blocks can be stacked to form a deep model (see, e.g., Thickstun, 2021).

Transformer block. The central operation of a transformer is self-attention (Vaswani et al., 2017). In the Pre-Layer Normalization (Pre-LN) setting (see Xiong et al., 2020), attention is applied to normalized token representations. For a token $\mathbf{x}_i$, a transformer block updates the representation via the scheme

$$\begin{aligned} \mathbf{u}'_i &= \text{Attn}(\text{LayerNorm}(\mathbf{x}_i; \boldsymbol{\gamma}_1, \boldsymbol{\beta}_1)), \\ \mathbf{u}_i &= \mathbf{x}_i + \mathbf{u}'_i, \\ \mathbf{z}'_i &= W_2^\top \sigma(W_1^\top \text{LayerNorm}(\mathbf{u}_i; \boldsymbol{\gamma}_2, \boldsymbol{\beta}_2)), \\ \mathbf{z}_i &= \mathbf{u}_i + \mathbf{z}'_i, \end{aligned} \quad (4)$$

where $W_1 \in \mathbb{R}^{k \times m}$ and $W_2 \in \mathbb{R}^{m \times k}$ are the weight matrices of a feedforward network (FFN), and $\sigma(\cdot)$ denotes a nonlinearity such as ReLU or GELU.

Layer normalization is applied element-wise to each token. For a vector $\mathbf{v} = (v_1, \dots, v_k) \in \mathbb{R}^k$, it is defined as

$$\text{LayerNorm}(\mathbf{v}) = \boldsymbol{\gamma} \odot \frac{\mathbf{v} - \mu(\mathbf{v})}{\sigma(\mathbf{v})} + \boldsymbol{\beta}, \quad (5)$$

where $\odot$ denotes element-wise multiplication and

$$\mu(\mathbf{v}) = \frac{1}{k} \sum_{j=1}^k v_j, \quad \sigma(\mathbf{v}) = \sqrt{\frac{1}{k} \sum_{j=1}^k (v_j - \mu(\mathbf{v}))^2}.$$

Here, $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^k$ are learnable parameters (see Xiong et al., 2020).

Self-attention. We now define the multi-head attention operator $\text{Attn}(\cdot)$. In the multi-head setting, the global embedding dimension is k , and each head operates in a subspace of dimension

$$d_k := \frac{k}{n_{\text{heads}}}.$$

For each head $h \in \{1, \dots, n_{\text{heads}}\}$, we operate on pre-normalized input representations $\tilde{\mathbf{x}}_i = \text{LayerNorm}(\mathbf{x}_i; \boldsymbol{\gamma}_1, \boldsymbol{\beta}_1)$. Query, key, and value vectors are computed as

$$Q^{(h)}(\tilde{\mathbf{x}}_i) = W_{h,q}^\top \tilde{\mathbf{x}}_i, \quad K^{(h)}(\tilde{\mathbf{x}}_i) = W_{h,k}^\top \tilde{\mathbf{x}}_i, \quad V^{(h)}(\tilde{\mathbf{x}}_i) = W_{h,v}^\top \tilde{\mathbf{x}}_i, \quad (6)$$

with

$$W_{h,q}, W_{h,k}, W_{h,v} \in \mathbb{R}^{k \times d_k}, \quad Q^{(h)}(\tilde{\mathbf{x}}_i), K^{(h)}(\tilde{\mathbf{x}}_i), V^{(h)}(\tilde{\mathbf{x}}_i) \in \mathbb{R}^{d_k}.$$

Attention weights are computed via scaled dot products followed by a softmax:

$$\alpha_{i,j}^{(h)} = \text{softmax}\left(\frac{\langle Q^{(h)}(\tilde{\mathbf{x}}_i), K^{(h)}(\tilde{\mathbf{x}}_j) \rangle}{\sqrt{d_k}}\right) = \frac{\exp(\langle Q^{(h)}(\tilde{\mathbf{x}}_i), K^{(h)}(\tilde{\mathbf{x}}_j) \rangle / \sqrt{d_k})}{\sum_{l=1}^n \exp(\langle Q^{(h)}(\tilde{\mathbf{x}}_i), K^{(h)}(\tilde{\mathbf{x}}_l) \rangle / \sqrt{d_k})}. \quad (7)$$

The head outputs are weighted sums of values, which are projected and aggregated across heads:

$$\text{Attn}(\tilde{\mathbf{x}}_i) = \sum_{h=1}^{n_{\text{heads}}} W_{c,h}^\top \left(\sum_{j=1}^n \alpha_{i,j}^{(h)} V^{(h)}(\tilde{\mathbf{x}}_j) \right), \quad W_{c,h} \in \mathbb{R}^{d_k \times k}. \quad (8)$$

Self-attention can thus be interpreted as a learned, data-dependent aggregation rule: each token representation is updated by mixing information from all other tokens, with weights determined by similarity in a learned space (see Nagler, 2023, Thickstun, 2021). This flexibility is a key factor behind the empirical success of transformers. A major limitation of full attention, however, is its quadratic computational and memory complexity in the sequence length n , since all pairwise similarities must be computed (see Zeng et al., 2025).

2.2 Prior-Data Fitted Networks (PFNs)

Prior-Data Fitted Networks (PFNs) (Müller et al., 2021) are a class of meta-learned predictors that shift part of the statistical workload from dataset-specific training to a large-scale pretraining phase. Instead of fitting a new model from scratch for each single dataset, a PFN is pretrained on a large collection of datasets drawn from a prescribed prior

over learning problems. At inference time, the pretrained model receives the observed dataset as input and directly produces predictions for new query points. In the PFN setting, no parameter update takes place during inference—the model adapts to the task in a single forward pass (see Müller et al., 2021).

PFNs achieve this by leveraging a form of in-context learning. In this respect, they are conceptually related to modern large language models, which can adapt their behavior from examples provided in the prompt without gradient-based fine-tuning (see Brown et al., 2020). For PFNs, the observed dataset itself serves as context: the model is given a set of labeled examples and is expected to infer predictions for new query points. PFNs also rely on the transformer architecture, though here it is not used for language modeling but is adapted to tabular prediction tasks. Thickstun (2021) interpret the transformer as a form of lookup table, in which query points are matched against key points.

Figure 1 illustrates how the input dataset is partitioned into a *context set* (labeled data, left) and a *query set* (unlabeled test point, indicated by a question mark). The arrows can be interpreted as lookup operations, which are implemented mathematically via self-attention. During pretraining, the model learns a task-dependent notion of similarity between data points, encoded in the attention mechanism (see Nagler, 2023). At inference time, this learned similarity measure is applied to relate the query point to the context set, enabling the model to produce predictions without additional optimization. Query points attend only to the observed context set, while no attention is allowed to other query points or their labels. This ensures that predictions are based solely on the labeled context data and is implemented through an appropriate attention mask (see Hollmann et al., 2023).

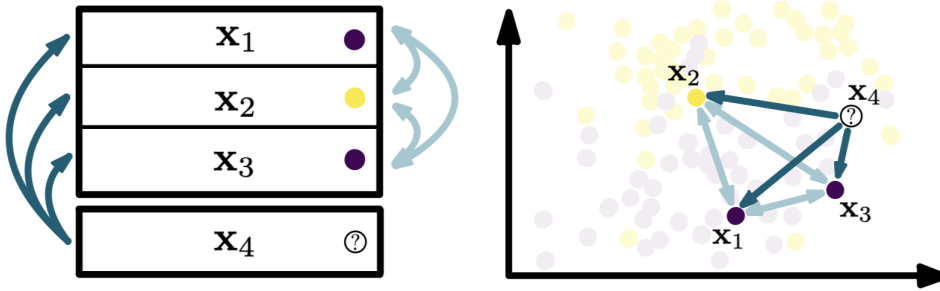


Figure 1: Conceptual visualization of a PFN, inspired by Müller et al. (2021). The figure is split into two parts: on the left, a small binary two-dimensional dataset with $n = 4$ is shown in schematic tabular form; on the right, the same data is represented in a coordinate system (axes omitted for clarity). Colors (yellow and purple) indicate class labels, while the test point $\mathbf{x}_4$ remains unlabeled (question mark). Light-blue arrows denote bidirectional attention among context points, allowing information exchange within the training set. Dark-blue arrows indicate attention from the query point to all context points. No attention is directed toward the query point.

This setup is particularly attractive in the small-data regime. Since the model has already been exposed to many related prediction tasks during pretraining, it can rely on this previously acquired statistical structure when only few observations are available at

test time (see Garnelo et al., 2018).

PFNs perform approximate Bayesian inference (see Müller et al., 2021). In Bayesian supervised learning, predictions for a new input $\mathbf{x}$ can be obtained via the posterior predictive distribution

$$p(y \mid \mathbf{x}, \mathcal{D}) = \int p(y \mid \mathbf{x}, \varphi) p(\varphi \mid \mathcal{D}) d\varphi, \quad (9)$$

where $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ denotes the observed supervised dataset, and φ represents latent model parameters or latent functions governing the data-generating mechanism. The posterior $p(\varphi \mid \mathcal{D})$ is obtained from a prior $p(\varphi)$ and the likelihood induced by the observed data. This integral is typically intractable for flexible model classes and is commonly approximated with methods such as Markov chain Monte Carlo or variational inference (Müller et al., 2021).

PFNs aim to bypass explicit posterior inference: by training directly on synthetic datasets drawn from a prior over latent data-generating mechanisms, the model learns to output predictive distributions that approximate the posterior predictive distribution.

2.2.1 Prior-fitting on synthetic datasets

PFNs are trained exclusively on synthetic data (Müller et al., 2021). Concretely, one specifies a family of data-generating mechanisms (DGMs)—for example linear models, polynomials, neural networks, or structural causal models—and a probability distribution over their parameters. Sampling from this distribution yields a stream of datasets with diverse characteristics (e.g., varying feature dimensionality, noise levels, and degrees of nonlinearity). In the original PFN work, Bayesian neural networks are used as DGMs: architectures and weights are sampled, inputs are generated at the root nodes, and labels are obtained by propagating inputs through the sampled network. This generates a diverse collection of datasets which are then used for pretraining. Later developments broaden this approach substantially, yielding priors that better reflect the heterogeneity of real tabular datasets (see Hollmann et al., 2025, Grinsztajn et al., 2025).

2.2.2 Training objective

Following Müller et al. (2021), the PFN training procedure can be described as follows.

Let q_θ denote a parameterized model that maps a context dataset and query inputs to predictive probabilities. At each training step, a dataset is sampled from the prior and split into a context part and a query part. Writing the context set as D and the query set as $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$, this can be expressed as

$$D \cup \{(\mathbf{x}_i, y_i)\}_{i=1}^m \sim p(\mathcal{D}). \quad (10)$$

The PFN is trained by minimizing the negative log-likelihood, referred to by Müller et al. (2021) as the Prior-Data Negative Log-Likelihood (Prior-Data NLL),

$$\ell(\theta) = \sum_{i=1}^m -\log q_\theta(y_i \mid \mathbf{x}_i, D). \quad (11)$$

Crucially, the loss is computed only on query points, while D serves as conditioning context information. The parameters θ are optimized using stochastic gradient descent over a large number of sampled datasets.

2.3 TabPFN

TabPFN is a specialization of the PFN framework for tabular machine learning (Hollmann et al., 2023). It addresses key challenges of tabular data, including heterogeneous feature types (continuous and categorical), missing values, varying numbers of features across datasets, and class imbalance, through a combination of synthetic pretraining priors and architectural adaptations. Since its original introduction, several versions of the model have been developed.

2.3.1 TabPFN-V1

TabPFN-V1 (Hollmann et al., 2023) adapts the original PFN approach to tabular classification and regression. Compared to the original PFN, it extends the training distribution to include unbalanced datasets and multiclass targets, and it introduces preprocessing and ensembling strategies to improve robustness. The model is designed to handle up to 1,000 training samples and 100 features. Since it is not inherently permutation-invariant with respect to feature order, TabPFN-V1 employs feature permutations (and other transformations) and averages predictions across multiple preprocessing variants. Moreover, variable feature dimensionality is handled by padding each input to a fixed maximum number of features (Hollmann et al., 2023). This naturally simplifies batching, as all datasets are mapped to a common input shape before being processed by the transformer layers. However, this also imposes a hard limit: datasets whose feature dimensionality exceeds this maximum cannot be processed without additional preprocessing, such as feature selection or truncation. From a practical perspective, this represents a notable limitation. Algorithmically, TabPFN-V1 remains close to the original PFN formulation: it operates on a two-dimensional token sequence representation, and predictions are produced via a single forward pass conditioned on the context set.

2.3.2 TabPFN-V2

TabPFN-V2 (Hollmann et al., 2025) builds on a substantially richer pretraining prior based on structural causal models (SCMs), complemented by post-processing transformations such as warping and quantization to better align synthetic and real-world tabular data distributions.

In addition, the architecture is revised to scale to larger datasets and to handle varying feature dimensionalities more naturally (Hollmann et al., 2025). Inspired by Lorch et al. (2022), the attention mechanism is factorized into two sublayers operating along different axes. Concretely, TabPFN-V2 alternates between *feature-wise* and *sample-wise* attention, enabling information exchange both across features and across data points. At inference time, the model is designed to handle up to 10,000 samples and 500 features.

As stated in Ye et al. (2025), the input table is first linearly encoded into a three-dimensional tensor

$$\mathbf{X} \in \mathbb{R}^{n \times (d+1) \times k}, \quad (12)$$

where each table cell is represented by a k -dimensional token $\mathbf{x}_{i,j} \in \mathbb{R}^k$. Feature-wise attention treats each row as a sequence of $(d+1)$ tokens and attends across columns, whereas sample-wise attention treats each column as a sequence of n tokens and attends across rows. By alternating these two operations, the model propagates information both across features and across samples (see Lorch et al., 2022). In the following, we formalize the transformer operations in the alternating-attention setting. To the best of our knowledge, an explicit formulation of this variant is not readily available in the existing literature; we therefore provide a formal specification, extending the standard notation with additional indices, while continuing to follow Thickstun (2021).

Transformer block. We employ the same pre-normalization architecture as in Section 2.1 and apply a FFN of the same form, but replace the single attention operator with a sequence of feature-wise and sample-wise attention, capturing dependencies along the feature and sample dimensions respectively. For each token $\mathbf{x}_{i,j}$, the block updates are given by

$$\begin{aligned} \mathbf{u}'_{i,j} &= \text{Attn}_{\text{feat}}(\text{LayerNorm}(\mathbf{x}_{i,j}; \boldsymbol{\gamma}_1, \boldsymbol{\beta}_1)), \\ \mathbf{u}_{i,j} &= \mathbf{x}_{i,j} + \mathbf{u}'_{i,j}, \\ \mathbf{v}'_{i,j} &= \text{Attn}_{\text{samp}}(\text{LayerNorm}(\mathbf{u}_{i,j}; \boldsymbol{\gamma}_2, \boldsymbol{\beta}_2)), \\ \mathbf{v}_{i,j} &= \mathbf{u}_{i,j} + \mathbf{v}'_{i,j}, \\ \mathbf{z}'_{i,j} &= W_2^\top \sigma(W_1^\top \text{LayerNorm}(\mathbf{v}_{i,j}; \boldsymbol{\gamma}_3, \boldsymbol{\beta}_3)), \\ \mathbf{z}_{i,j} &= \mathbf{v}_{i,j} + \mathbf{z}'_{i,j}. \end{aligned} \quad (13)$$

We now define the attention operators $\text{Attn}_{\text{feat}}$ and $\text{Attn}_{\text{samp}}$. Feature-wise and sample-wise attention employ separate sets of projection and output matrices, i.e., all parameters are independent across the two attention steps.

Feature-wise attention. Feature-wise attention operates on pre-normalized input representations $\tilde{\mathbf{x}}_{i,j} = \text{LayerNorm}(\mathbf{x}_{i,j}; \boldsymbol{\gamma}_1, \boldsymbol{\beta}_1)$ (as defined in Section 2.1). For each head $h \in \{1, \dots, n_{\text{heads}}\}$, query, key, and value vectors are computed as

$$Q^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}) = W_{h,q}^{(\text{feat})\top} \tilde{\mathbf{x}}_{i,j}, \quad K^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}) = W_{h,k}^{(\text{feat})\top} \tilde{\mathbf{x}}_{i,j}, \quad V^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}) = W_{h,v}^{(\text{feat})\top} \tilde{\mathbf{x}}_{i,j},$$

with

$$W_{h,q}^{(\text{feat})}, W_{h,k}^{(\text{feat})}, W_{h,v}^{(\text{feat})} \in \mathbb{R}^{k \times d_k}, \quad Q^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}), K^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}), V^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}) \in \mathbb{R}^{d_k}.$$

Attention is applied across the feature dimension while keeping the sample index fixed. The attention weights are

$$\begin{aligned} \alpha_{i,j,j'}^{(h,\text{feat})} &= \text{softmax}\left(\frac{\langle Q^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}), K^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j'}) \rangle}{\sqrt{d_k}}\right) \\ &= \frac{\exp(\langle Q^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}), K^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j'}) \rangle / \sqrt{d_k})}{\sum_{l=1}^{d+1} \exp(\langle Q^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j}), K^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,l}) \rangle / \sqrt{d_k})}, \quad j' \in \{1, \dots, d+1\}. \end{aligned} \quad (14)$$

The multi-head feature-wise attention output at position (i, j) is then given by

$$\text{Attn}_{\text{feat}}(\mathbf{x}_{i,j}) = \sum_{h=1}^{n_{\text{heads}}} W_{c,h}^{(\text{feat})\top} \left(\sum_{j'=1}^{d+1} \alpha_{i,j,j'}^{(h,\text{feat})} V^{(h,\text{feat})}(\tilde{\mathbf{x}}_{i,j'}) \right), \quad W_{c,h}^{(\text{feat})} \in \mathbb{R}^{d_k \times k}. \quad (15)$$

Sample-wise attention. Analogously, sample-wise attention operates on the feature-updated representations using independent parameters. As input, we take the pre-normalized feature representations $\tilde{\mathbf{u}}_{i,j} = \text{LayerNorm}(\mathbf{u}_{i,j}; \gamma_2, \beta_2)$. For each head $h \in \{1, \dots, n_{\text{heads}}\}$, query, key, and value vectors are computed as

$$Q^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}) = W_{h,q}^{(\text{samp})\top} \tilde{\mathbf{u}}_{i,j}, \quad K^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}) = W_{h,k}^{(\text{samp})\top} \tilde{\mathbf{u}}_{i,j}, \quad V^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}) = W_{h,v}^{(\text{samp})\top} \tilde{\mathbf{u}}_{i,j},$$

with

$$W_{h,q}^{(\text{samp})}, W_{h,k}^{(\text{samp})}, W_{h,v}^{(\text{samp})} \in \mathbb{R}^{k \times d_k}, \quad Q^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}), K^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}), V^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}) \in \mathbb{R}^{d_k}.$$

Attention is applied across the sample dimension while keeping the feature index fixed. The attention weights are

$$\begin{aligned} \alpha_{i,j,i'}^{(h,\text{samp})} &= \text{softmax} \left(\frac{\langle Q^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}), K^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i',j}) \rangle}{\sqrt{d_k}} \right) \\ &= \frac{\exp(\langle Q^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}), K^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i',j}) \rangle / \sqrt{d_k})}{\sum_{l=1}^n \exp(\langle Q^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i,j}), K^{(h,\text{samp})}(\tilde{\mathbf{u}}_{l,j}) \rangle / \sqrt{d_k})}, \quad i' \in \{1, \dots, n\}. \end{aligned} \quad (16)$$

The multi-head sample-wise attention output at position (i, j) is then given by

$$\text{Attn}_{\text{samp}}(\mathbf{u}_{i,j}) = \sum_{h=1}^{n_{\text{heads}}} W_{c,h}^{(\text{samp})\top} \left(\sum_{i'=1}^n \alpha_{i,j,i'}^{(h,\text{samp})} V^{(h,\text{samp})}(\tilde{\mathbf{u}}_{i',j}) \right), \quad W_{c,h}^{(\text{samp})} \in \mathbb{R}^{d_k \times k}. \quad (17)$$

The attention mechanism remains conceptually analogous to that introduced in Section 2.1, but is now applied separately along the two table axes. As in Section 2.1, the tensor shape is preserved, i.e., the block maps $\mathbb{R}^{n \times (d+1) \times k}$ to itself. Figure 2 illustrates the overall data flow of this alternating-attention architecture.

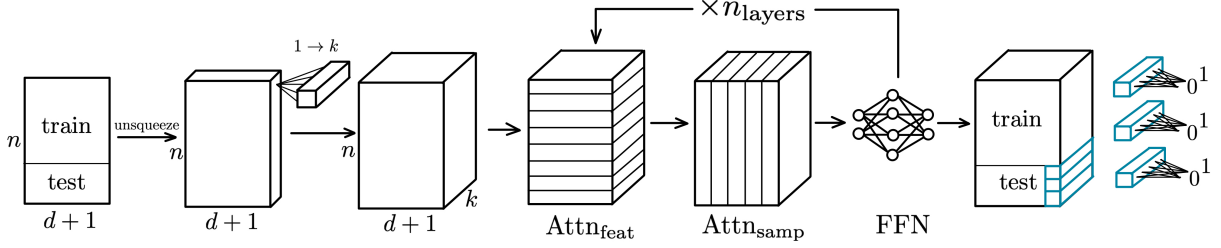


Figure 2: Overview of the alternating-attention architecture, inspired by Ye et al. (2025). The input dataset is first encoded into a three-dimensional tensor in $\mathbb{R}^{n \times (d+1) \times k}$. This representation is processed by a stack of n_{layers} transformer blocks, each consisting of feature-wise attention, sample-wise attention, and a feedforward network. While not shown explicitly, layer normalization is included with each sublayer. After the final block, the enriched token representations corresponding to the last column (i.e., the $(d+1)$ -th feature) are extracted and mapped to output logits via a multilayer perceptron (MLP), illustrated here for a binary prediction task.

Architectural limitations. This revised architecture no longer requires fixing a maximum number of features as in TabPFN-V1. Instead, the feature dimension is modeled as a variable-length sequence, analogous to the sample dimension, allowing for more flexible handling of heterogeneous tabular inputs (Hollmann et al., 2025). However, this flexibility comes at an additional computational cost. This means that each V2 Transformer block contains two separate attention modules—one feature-wise and one sample-wise—whereas V1 uses only a single attention operation. Each of these attention modules introduces its own query, key, value, and output projections of size $k \times k$. As a consequence, the attention-related parameter count scales quadratically in the embedding dimension k , and the alternating design effectively duplicates this contribution up to constant factors.

Second, the computational burden of self-attention remains challenging because its cost scales quadratically with the sequence length. In TabPFN-V2, the sample-wise attention step is often the main bottleneck, as it operates over sequences of length n for each of the $(d+1)$ columns. Consequently, the corresponding attention tensors scale as $(d+1) \times n \times n$, before accounting for additional batch dimensions. In our experiments, this leads to a substantial increase in memory usage and overall compute when combined with batched training and large synthetic datasets, a limitation that has also been noted in e.g. Qu et al. (2025).

One might expect that, since the feature dimension is processed analogously to the sample dimension, the resulting model would also be permutation invariant with respect to feature order. However, as noted by Hollmann et al. (2025), this is not the case. A plausible reason is that features are not treated as anonymous tokens: each feature column receives an additional feature-specific perturbation term (see Ye et al., 2025). Concretely, the representation of the j -th feature contains an offset r_j , which depends on its column index. As a result, permuting the feature order changes the input representation seen by the model, and therefore does not, in general, leave the prediction unchanged. Consequently, Hollmann et al. (2025) adopt the same ensembling strategy as Hollmann et al. (2023), shuffling feature order across ensemble members.

At the same time, the architecture provides useful opportunities for exploratory analysis of the model’s internal representations. For example, one can examine how attention is distributed across columns on average and relate this to ground-truth feature importance. In addition, as in Ye et al. (2025), it is possible to visualize how the latent representations of individual table cells evolve over the depth of the network. Their analysis further suggests that TabPFN-V2 can internalize meaningful relationships between features directly through in-context learning, even though the input embeddings themselves are randomized. As these aspects are not part of the main narrative of this thesis, they are discussed only briefly in Appendix A.1, which contains two exploratory experiments with corresponding visualizations.

2.3.3 TabPFN-V2.5

TabPFN-2.5 (see Grinsztajn et al., 2025) continues this line of development with a stronger emphasis on usability, scalability, and deployment efficiency. Beyond architectural refinements, the model is trained with a broader pretraining prior, has been scaled to a deeper architecture, and is designed to handle substantially larger datasets than its predecessors, supporting up to 50,000 samples and 2,000 features. Empirically, the model also represents a clear performance advance: the authors report that TabPFN-2.5 matches the accuracy of AutoGluon 1.4—a highly optimized and computationally expensive four-hour ensemble that even includes TabPFN-V2. In addition, Grinsztajn et al. (2025) place particular emphasis on faster inference and practical integration into tabular machine-learning workflows. One direction is model distillation: the in-context predictor can be transformed into more conventional model classes, such as MLPs or tree ensembles, which are easier to export and deploy while preserving most of the original performance. An overview of the different TabPFN versions and their main differences is provided in the appendix.

2.4 Flow Matching

In the original TabPFN framework, regression is handled by discretizing the PPD into bins and treating the problem analogously to classification over a finite support; see Chapter 4.1 for details. In this work we present a fundamentally different approach: We model the PPD in a fully generative way, such that one can draw samples from it directly rather than first instantiating it on a fixed grid. To this end, we make use of *Flow Matching* (see Lipman et al., 2022).

More generally, the goal is to draw samples from an unknown target distribution p_{data} . In the present context, this target distribution is the PPD, although the following formulation is more general. Flow Matching achieves this by learning a time-dependent *velocity field* that transports samples from a simple base distribution (e.g. standard Gaussian noise) toward the target distribution (see Lipman et al., 2022).

Following Lipman et al. (2022), let c be the dimensionality of p_{data} and consider a time-dependent *flow* $\psi : [0, 1] \times \mathbb{R}^c \rightarrow \mathbb{R}^c$, defined as the solution of the ordinary differential equation (ODE)

$$\frac{d}{dt}\psi_t(y) = u_t(\psi_t(y)), \quad \psi_0(y) = y. \quad (18)$$

Here, $u_t : \mathbb{R}^c \rightarrow \mathbb{R}^c$ denotes a time-dependent vector field, i.e. the velocity field. For every time t and position y , $u_t(y)$ specifies the instantaneous direction and magnitude of motion at that point. While u_t describes a time-dependent motion trajectory, the flow ψ_t can be thought of as positions of particles at each time step.

We initialize the system by sampling

$$Y_0 \sim p_0 = \mathcal{N}(0, I_c), \quad (19)$$

where p_0 is a simple base distribution. The flow then transports this initial random variable along the ODE trajectory, so that

$$Y_t := \psi_t(Y_0). \quad (20)$$

If the velocity field u_t is chosen appropriately, the distribution of Y_t evolves over time according to a time-dependent density p_t , called a *probability path*. In particular, if training is successful, the final distribution satisfies

$$Y_1 \sim p_{\text{data}}.$$

To build intuition, Figure 3 visualizes the end goal of the transport process in our setting. The left panel shows the evolution of samples under the learned transport. The right panel shows the resulting PPD embedded in the data context at the corresponding test input.

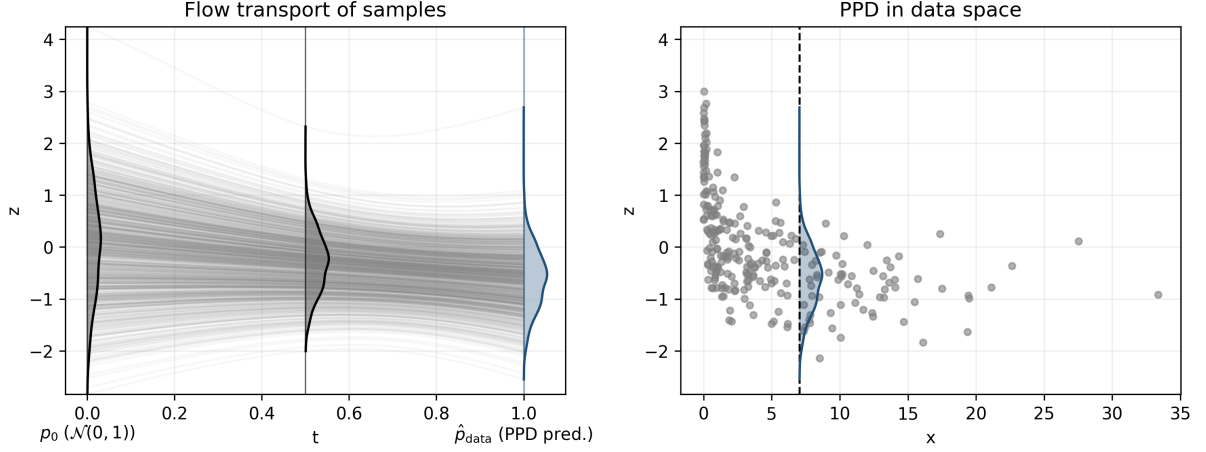


Figure 3: Visualization of the flow-based transport underlying Flow Matching for a single query point. Left: Evolution of samples in latent (z) space over time. For a fixed test input, 1000 samples are drawn from the base distribution $p_0 = \mathcal{N}(0, 1)$ at $t = 0$. Each trajectory shows how an individual sample is transported by the learned vector field. At $t = 0.5$, an intermediate distribution is illustrated, and at $t = 1$ the final distribution (blue) represents the learned predictive distribution $\hat{p}_{\text{data}}$. Right: The same final distribution is embedded into the data context at the corresponding test input location. While the horizontal axis represents the feature space, the vertical axis remains in latent (z) space, reflecting the internal representation used by the model.

We now turn our attention to how such a velocity field can be learned. For this purpose we adapt the notation of Lipman et al. (2022) and Holderrieth and Erives (2025). The theoretical Flow Matching objective compares a learned velocity field $u_{\theta,t}$ to a target vector field u_t^{target} under the intermediate density p_t :

$$\ell_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}[0,1], y_t \sim p_t} \left[\|u_{\theta,t}(y_t) - u_t^{\text{target}}(y_t)\|_2^2 \right]. \quad (21)$$

In practice, both the marginal density p_t and the corresponding marginal target field are intractable. To circumvent this issue, Lipman et al. (2022) propose *Conditional Flow Matching (CFM)*, which constructs probability paths conditioned on individual data samples and shows that optimizing the conditional objective yields identical gradients to the original Flow Matching objective.

A particularly simple and effective (see Holderrieth and Erives, 2025) choice is the *Gaussian Conditional Optimal Transport (CondOT) path*. Let $y \sim p_{\text{data}}$ be a data sample and $z \sim \mathcal{N}(0, I_c)$ a noise sample. For $t \sim \text{Unif}[0, 1]$ we define the interpolated point

$$y_t = ty + (1 - t)z. \quad (22)$$

For this construction, the corresponding conditional target velocity is

$$u_t^{\text{target}}(y_t) = y - z, \quad (23)$$

i.e. the time-independent difference between the target value and the noise sample.

This yields the (Gaussian) *Conditional optimal-transport Flow Matching* objective

$$\ell_{\text{CFM}}^{\text{OT,Gauss}}(\theta) = \mathbb{E}_{t \sim \text{Unif}[0,1], y \sim p_{\text{data}}, z \sim \mathcal{N}(0, I_c)} \left[\|u_{\theta,t}(ty + (1-t)z) - (y - z)\|_2^2 \right]. \quad (24)$$

The above objective can readily be used for training. Algorithm 1 shows a general training pseudo code for training a Flow Matching model (similar to Holderrieth and Erives, 2025).

Algorithm 1: Flow Matching training algorithm (Gaussian CondOT), batch size 1

Input: Samples from the target distribution $y \sim p_{\text{data}}$, untrained neural network u_{θ}

for *each training step* **do**

- Sample $t \sim \text{Unif}[0, 1]$, $z \sim \mathcal{N}(0, I_c)$ pick one y from the target samples;
- Interpolate $y_t \leftarrow t y + (1 - t) z$;
- Compute $\ell(\theta) \leftarrow \|u_{\theta,t}(y_t) - (y - z)\|_2^2$;
- Update θ by gradient descent on $\ell(\theta)$;

In the PFN setting, the velocity field is additionally conditioned on a dataset context and a query input, which we denote by $u_{\theta}(\mathbf{x}, D, y_t, t)$. Details on the algorithm will be discussed in Section 4.3.

3 Related Work

Classical models for tabular data. Tabular prediction problems are ubiquitous, and for many years the state of the art has been dominated by tree-based methods, in particular gradient-boosted decision trees such as XGBoost, LightGBM, and CatBoost (Ye et al., 2025). A common explanation is that tabular datasets are highly heterogeneous: they mix continuous and categorical variables, exhibit strong feature interactions, and often come with irregular scaling and missingness patterns (see Hollmann et al., 2025). In contrast, standard deep learning architectures tend to be less robust in this regime (Hollmann et al., 2025). While gradient boosting is therefore a strong default, it also comes with limitations: models are trained per dataset and do not naturally transfer knowledge across tasks, and they are not as easily integrated into end-to-end differentiable pipelines as neural networks (Hollmann et al., 2025).

Deep learning architectures tailored to tables. Motivated by these shortcomings, a large body of work adapts deep learning ideas to the tabular setting. One direction builds differentiable analogues of decision trees. NODE (Neural Oblivious Decision Ensembles) uses *oblivious* trees as trainable building blocks—trees where all nodes at the same depth share the splitting feature and threshold—allowing end-to-end optimization with gradient-based methods and enabling deeper stacked ensembles (see Popov et al., 2019). Another line of work focuses on feature selection and sparse computation. TabNet (Arik and Pfister, 2021) employs a sequential, mask-based attention mechanism to select features at each decision step, providing a form of interpretability, although its “attention” differs from transformer-style self-attention. A third direction adapts transformer architectures to represent tables. TabTransformer (Huang et al., 2020) contextualizes categorical features via self-attention while treating numerical features more conventionally, which can limit interaction modeling across feature types. SAINT (Somepalli et al., 2021) addresses this by embedding both categorical and continuous features into a shared representation space and combining attention across features within a row with attention across rows (intersample attention), effectively introducing an alternating factorization of attention for tables. Similarly, FT-Transformer (Gorishniy et al., 2021) proposes a streamlined transformer backbone with dedicated tokenization for mixed feature types and demonstrates competitive performance across a wide range of tabular benchmarks.

Foundation models for tabular data. Foundation models have so far been most prominent in domains such as natural language processing and computer vision (Zhou et al., 2025, Kirillov et al., 2023). For tabular data, however, comparable large-scale pretrained models emerged only more recently (Van Breugel and Van Der Schaar, 2024). One natural approach is to leverage large language models and their in-context learning capabilities by linearizing tables into text. TaBERT (Yin et al., 2020), for example, learns joint representations of tables and their associated textual context. Other works directly prompt LLMs with tabular data serialized as strings, often together with natural-language descriptions, and report strong performance in certain settings (Hegselmann et al., 2023). However, representing tables as text can be inefficient: the token budget grows quickly with the number of rows and columns, making long-context limitations a

practical bottleneck (Ma et al., 2025).

Scaling in-context learning for large tables. TabPFN-style methods cast supervised learning on small tabular datasets as an in-context learning problem and rely on pretraining over synthetic tasks. Building on this idea, TabICL introduces a two-stage architecture explicitly designed to scale to much larger datasets, reaching up to hundreds of thousands of rows and hundreds of features (Qu et al., 2025). Its main strategy is to compress the column dimension before performing context-aware interactions, combining distribution-aware column embeddings, which capture per-feature statistics, with row-wise interaction modules that model dependencies across samples. TabICL-v2 extends this line of work through an improved synthetic pretraining pipeline and several architectural refinements, including feature grouping mechanisms and modifications to the attention and normalization components, in order to improve generalization on larger tables (Qu et al., 2026, Qiu et al., 2025).

Other extensions directly address a common limitation of standard attention, namely its quadratic scaling in the sequence length. One prominent example is linear attention (see Zeng et al., 2025). Instead of explicitly computing all pairwise query-key interactions followed by a softmax, linear attention replaces this similarity computation with a kernel feature map. This allows global summary statistics over keys and values to be computed once and then combined with each query individually, thereby reducing the computational complexity from quadratic to linear in the sequence length.

Other approaches aim to reduce the effective context size itself. This can be done, for example, through boosting-like procedures that iteratively refine a smaller context (see Wang et al., 2025), or through prompt-optimization methods that explicitly optimize which context information is presented to the model (see Feuer et al., 2024). A different strategy is to introduce a separate data encoder that maps the input data into the representation space most suitable for the PFN (see Liu and Ye, 2025). In this setup, the PFN itself remains frozen, while only the encoder is fine-tuned so that the data representations become better aligned with the pretrained model.

Extensions and adaptations of TabPFN. As highlighted by Grinsztajn et al. (2025), TabPFN has already given rise to a broad range of extensions tailored to more specialized use cases. Examples include adaptations for time-series forecasting (Hoo et al., 2024), reinforcement learning (Schiff et al., 2025), and causal inference (see, e.g. Balazadeh et al., 2025). Beyond such methodological extensions, TabPFN has also been applied directly across a wide range of domains, including healthcare, finance, and manufacturing (Grinsztajn et al., 2025).

The present thesis is situated most directly within the TabPFN line of work and contributes to it by studying the interaction between prior design and architecture, as well as by proposing new regression extensions.

4 PPD Modelling for Regression

In the classification setting it is straightforward to model the PPD: the distribution is discrete and can be represented exactly by a finite set of probabilities (e.g., in the binary case by $p(y = 1 \mid \mathbf{x}, \mathcal{D})$ and its complement). The question is how to represent an entire *continuous* predictive distribution in regression.

4.1 How TabPFN models the PPD

TabPFN follows a “regression via classification” paradigm of Torgo and Gama (1997) and discretizes the target space into intervals, turning regression into a multiclass classification problem (see Müller et al., 2021, Hollmann et al., 2025). Training then minimizes a cross-entropy objective for the event that a target value falls into a particular interval. Müller et al. (2021) formalize this construction as a *Riemann distribution*: the softmax outputs approximate bin masses, which correspond to a piecewise-constant density after dividing by the respective bin widths. The bin boundaries are chosen *a priori* by drawing a large sample from the target prior (before training) and selecting a fixed number of “buckets” such that each bucket contains approximately equal prior probability mass. For priors with unbounded support, the outermost buckets can be replaced by appropriately scaled parametric tails (e.g., half-normal components), yielding a hybrid distribution that combines discretization in the bulk with parametric behavior in the extremes (see Müller et al., 2021).

This discretization-based approach comes with natural limitations. First, standard cross-entropy treats buckets as independent classes and does not encode their inherent ordering: confusing a neighboring bucket is penalized just as strongly as confusing a far-away bucket. Torgo and Gama (1997) therefore suggest incorporating an explicit penalty structure that reflects the ordinal nature of the bins. However the implementation of TabPFN does not seem to use this cost-based approach. Second, discretization introduces a discretization error because all information within a bucket is collapsed; fine-grained variation inside an interval is not represented. TabPFN mitigates this effect by using a large number of buckets (5000) (see Hollmann et al., 2025).

In this work we present two alternative approaches to model the continuous PPD in regression tasks: 1. We model the output distribution as a B-spline. 2. We turn the model into a generative model, leveraging Flow Matching to directly sample from the PPD, allowing for Monte Carlo approximation.

4.2 PPD as a B-spline

As described above, in the original TabPFN regression setup, the PPD is represented by a bucketized output space. Let $\{\text{bucket}_j\}_{j=1}^J$ denote a partition of the target space. Then the predictive distribution can be written as

$$p_{\theta}(y \mid \mathbf{x}, D) = \sum_{j=1}^J \pi_{\theta,j}(\mathbf{x}, D) \mathbf{1}(y \in \text{bucket}_j),$$

where $\pi_{\theta,j}(\mathbf{x}, D)$ denotes the predicted probability assigned to bucket j . This representation is already equivalent to a B-spline basis expansion (see De Boor, 1986) of degree zero, since the model is piecewise constant over the predefined intervals.

A natural extension is therefore to replace the indicator basis by a more general spline basis. In particular, one may write

$$p_{\theta}(y \mid \mathbf{x}, D) = \sum_{j=1}^M \pi_{\theta,j}(\mathbf{x}, D) \mathcal{B}_j(y), \quad (25)$$

where $\mathcal{B}_j(y)$ denotes the j -th B-spline basis function and $\pi_{\theta,j}(\mathbf{x}, D)$ are non-negative mixture weights satisfying

$$\sum_{j=1}^M \pi_{\theta,j}(\mathbf{x}, D) = 1.$$

Figure 4 contrasts the two bucket-based modeling approaches considered in this thesis. On the left, the original TabPFN-style construction corresponds to a piecewise-constant density, which can equivalently be viewed as a spline basis of degree 0. On the right, the spline-based variant with degree 3 yields overlapping basis functions and thus induces a smoother local interpolation structure. In contrast to the bucketized formulation, neighboring spline components are no longer independent but interact through their overlap. As the spline degree increases, the basis functions become wider and span more neighboring intervals. One can also observe that the intervals become wider farther away from 0. This is a consequence of the predefined bucket boundaries: they were chosen to be approximately equal-mass (see Müller et al., 2021) under the marginal target distribution in z -space, rather than equally spaced.

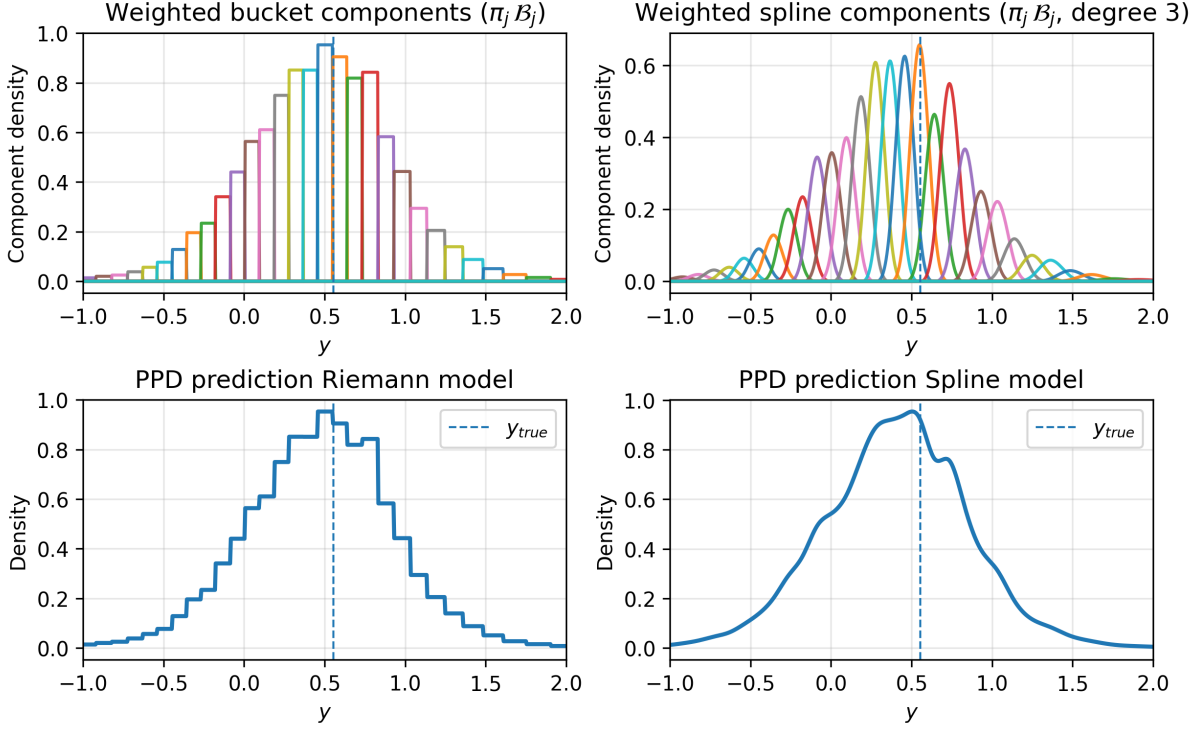


Figure 4: Comparison of the basis functions underlying two PPD-modeling approaches: the original TabPFN-style piecewise-constant formulation (left), corresponding to degree-0 splines, and the smoother spline-based formulation with degree 3 (right).

A convenient feature of the spline approach is that the overall model architecture remains unchanged relative to the Riemann-style bucket model. In particular, the network still outputs raw logits, which are transformed via a softmax into mixture weights $\pi_{\theta,j}(x, D)$. The main changes occur in the training procedure. Instead of assigning each target value to a single bucket, the target is evaluated under the spline basis expansion, and the resulting predictive density is trained via a continuous negative log-likelihood,

$$\ell(\theta) = - \sum_{i=1}^m \log p_{\theta}(y_i \mid \mathbf{x}_i, D),$$

where $i = 1, \dots, m$ indexes the elements of the test set. Using the spline-based predictive density from Equation (25), the training objective becomes

$$\ell(\theta) = - \sum_{i=1}^m \log \left(\sum_{j=1}^M \pi_{\theta,j}(\mathbf{x}_i, D) \mathcal{B}_j(y_i) \right).$$

The density components $\mathcal{B}_j$ are given by normalized B-spline basis functions of degree p , obtained from the Cox–de Boor recursion (see De Boor, 1986). Given a knot vector $\tau_0, \dots, \tau_L$, the degree-zero basis is defined as

$$N_{r,0}(y) = \begin{cases} 1, & \tau_r \leq y < \tau_{r+1}, \\ 0, & \text{otherwise,} \end{cases}$$

and higher-degree basis functions are obtained recursively via

$$N_{r,p}(y) = \frac{y - \tau_r}{\tau_{r+p} - \tau_r} N_{r,p-1}(y) + \frac{\tau_{r+p+1} - y}{\tau_{r+p+1} - \tau_{r+1}} N_{r+1,p-1}(y).$$

with the usual convention that terms with zero denominator are treated as zero. Since these basis functions are non-negative and locally supported, they are well suited as density components after appropriate normalization.

Another practical advantage is that the bucket borders determined a priori for the Riemann model can be reused directly as knot locations. Since these borders were chosen more densely in high-probability regions around the expectation and more sparsely in the tails, they already provide an adaptive discretization of the target space. As a consequence, the spline basis inherits a higher effective resolution where the predictive density is expected to concentrate most of its mass, while maintaining a coarser representation in low-density tail regions.

Increasing the spline degree changes the number of basis functions. While the degree-zero case corresponds exactly to the original number of buckets, higher degrees require repeated boundary knots in the clamped knot vector, which increases the number of basis functions without changing the underlying bucket intervals. Intuitively, this happens because higher-degree boundary basis functions must remain well-defined despite having support truncated at the edges of the domain, so the spline basis needs a slightly expanded representation near the boundaries.

4.3 PPD-Modeling via Flow Matching

Section 2.4 introduces *Flow Matching* as a generative approach for sampling from an unknown target distribution. In this setting, the model no longer outputs a discretized PPD, but instead learns a *velocity field*

$$u_\theta(\mathbf{x}, D, y_t, t),$$

which governs how samples are transported over time. In the PFN setting, the target sample y corresponds to the label of a test input, which we denote by y_{test} . During training, we continue to sample synthetic datasets as before, but now additionally draw a random time point $t \in [0, 1]$, for instance from a standard uniform distribution, as well as a noise sample z from a standard normal distribution. As described in Section 2.4, this allows us to construct an interpolated point

$$y_t = t y_{test} + (1 - t) z,$$

between a noise sample z and the target value y_{test} . The corresponding target velocity is then given by $y_{test} - z$, which defines the regression target used in the Flow Matching objective. This in turn allows the loss to be computed and optimized via gradient descent. We adapt Algorithm 1 from Section 2.4 for the flow-based PFN training. The simplified training procedure is summarized in Algorithm 2.

Algorithm 2: Flow-based PFN training algorithm, batch size 1

Input: Untrained neural network u_θ , prior over datasets $p(\mathcal{D})$

for each training step do

Sample $D \cup \{(\mathbf{x}, y_{test})\} \sim p(\mathcal{D})$;
 Sample $t \sim \text{Unif}[0, 1]$, $z \sim \mathcal{N}(0, 1)$;
 Interpolate $y_t \leftarrow t y_{test} + (1 - t) z$;
 Compute $\ell(\theta) \leftarrow \|u_\theta(\mathbf{x}, D, y_t, t) - (y_{test} - z)\|_2^2$;
 Update θ by gradient descent on $\ell(\theta)$;

At inference time, the dataset-dependent context representation only needs to be computed once at the beginning, since it remains fixed throughout the transport process. Sampling then starts at time $t = 0$ from an initial point (see Holderrieth and Erives, 2025)

$$y_0 \sim \mathcal{N}(0, 1),$$

and the learned velocity field is used to iteratively propagate this sample toward $t = 1$. In the continuous formulation, this corresponds to solving the ordinary differential equation

$$\frac{dy_t}{dt} = u_\theta(\mathbf{x}, D, y_t, t).$$

In practice, this ODE must be approximated numerically. In this thesis, we use the explicit Euler method (see Holderrieth and Erives, 2025). To this end, the interval $[0, 1]$ is discretized into S equidistant steps with step size

$$\Delta t = \frac{1}{S}.$$

We use $s = 0, \dots, S$ to index the resulting discrete time points, while t continues to denote continuous time. The corresponding discrete time grid is given by

$$t_s = s \Delta t.$$

The Euler update then takes the form (see Holderrieth and Erives, 2025)

$$y_{s+1} = y_s + \Delta t u_\theta(\mathbf{x}, D, y_s, t_s), \quad s = 0, \dots, S - 1.$$

Thus, y_s denotes the discrete numerical approximation to the continuous trajectory y_t at time t_s . Other numerical integration schemes, such as higher-order ODE solvers, could also be used, but Euler integration is sufficient here and keeps the sampling procedure simple. The resulting iterative sampling procedure is summarized in Algorithm 3.

Algorithm 3: Sampling from the PPD via Flow Matching using explicit Euler integration

Input: Unlabeled query input $\mathbf{x}$, observed dataset D , trained velocity model u_θ ,
number of integration steps S
Fix context $(\mathbf{x}, D)$;
Sample initial state $y_0 \sim \mathcal{N}(0, 1)$;
Set $\Delta t \leftarrow 1/S$;
for $s = 0, \dots, S - 1$ **do**
 Set $t_s \leftarrow s \Delta t$;
 Compute velocity $v_s \leftarrow u_\theta(\mathbf{x}, D, y_s, t_s)$;
 Update state $y_{s+1} \leftarrow y_s + \Delta t v_s$;
end for
Return final sample y_S ;

What remains is the practical question of how to incorporate the additional inputs t and y_t into the model so that it can generate samples from the PPD. The architectural details of the resulting implementation are discussed in Subsection 5.1.

5 Experiments

A central focus of the experimental part of this thesis is the role of the *pretraining prior* and the *model architecture*, and how their interaction influences downstream performance. In addition, we evaluate the two newly proposed approaches for regression, namely modeling the PPD via B-splines and the generative Flow Matching approach.

To address these questions, we first discuss the core architectural and implementation details of the models in Subsection 5.1. Subsection 5.2 then introduces the two pretraining priors of differing complexity that are used throughout the experiments. In Subsection 5.3, we conduct a random hyperparameter search for binary classification in order to better understand how architectural choices affect training behavior. We also examine the relationship between training performance, convergence behavior, and generalization. Building on these findings, we conduct a benchmark for both binary classification and regression in Section 5.4, comparing the prototypes to the publicly available TabPFN-V2 models as baselines.

All implementations are based on `PyTorch` (Paszke et al., 2019), with `TabPFN` (Hollmann et al., 2025) serving as the primary baseline. Data handling and analysis rely on `pandas` (Wes McKinney, 2010), `NumPy` (Harris et al., 2020), `SciPy` (Virtanen et al., 2020), `scikit-learn` (Kramer, 2016), and `statsmodels` (Seabold et al., 2010), while visualizations were produced using `matplotlib` (Hunter, 2007) and `seaborn` (Waskom, 2021).

5.1 Model Implementation

In total, five models are implemented in this work. An overview is provided in Table 1. For the implementation of the alternating-attention scheme of V2, Pfefferle et al. (2025) and Ye et al. (2025) served as methodological references. While Pfefferle et al. (2025) employs post-layer normalization, the models presented here adopt pre-layer normalization, consistent with Section 2.1. In the binary classification setting, both a V1 and a V2 variant were implemented to support initial comparative experiments; however, the subsequent analysis focuses exclusively on V2.

In contrast to Hollmann et al. (2023, 2025), Grinsztajn et al. (2025), no ensembling is applied in the prototype models presented in this work.

Setting	Name	Attention	Head mapping
Classification	TabPFN-V1	1x 2D attention	$k \rightarrow 2$ (binary)
	TabPFN-V2	2x 1D attention	$k \rightarrow 2$ (binary)
Regression	Riemann model	2x 1D attention	$k \rightarrow 50$ (50 bins)
	Spline model	2x 1D attention	$k \rightarrow 50$ (50 bins)
	FlowPFN	2x 1D attention	$k \rightarrow 1$ (velocity scalar)

Table 1: Overview of the implemented models. Here, “1x 2D attention” denotes a single two-dimensional attention operation, whereas “2x 1D attention” denotes alternating-attention over the feature and sample dimensions. Riemann model refers to the implementation close to the original regression model, with piecewise constant indicator functions for the bins. For the Riemann and spline models, the PPD is discretized into 50 bins. In the original work, the predictive distribution is discretized into 5000 bins (see Hollmann et al., 2025).

Encoder. All models use the same internal preprocessing pipeline. Upon ingestion, inputs are first standardized into z -space. To improve training stability, a soft-clipping step is then applied to outliers in z -space; in the regression setting, this is also applied to the target variable. This is conceptually related to one of the preprocessing steps described in Hollmann et al. (2025). However, that work employs a substantially more elaborate preprocessing pipeline, including SVD compression, power transforms, quantile-based spacing, and ensembling across preprocessing variants, which makes it difficult to isolate the contribution of any single component. The more recent TabPFN-V2.5 (Grinsztajn et al., 2025) is described more succinctly and does not document its preprocessing pipeline in comparable detail, although it is plausible that a similarly rich procedure is still used. In contrast, we deliberately keep preprocessing minimal: standardization, followed by soft-clipping via $\tanh$, and then a single-layer MLP encoder. While TabPFN-V2.5 uses a two-layer MLP encoder, we do not adopt this modification.

Backbone. The backbone consists of stacked transformer blocks. In the V2-based prototypes, each block contains a feature-attention layer, a sample-attention layer, and a feedforward block, all equipped with residual connections and layer normalization. The V1 prototype, by contrast, uses a single two-dimensional attention operation instead of the alternating-attention scheme.

Output heads. In the classification setting, the k -dimensional token of y_{test} is mapped directly to a two-dimensional logit vector via a linear layer. The regression models based on discretizing the predictive distribution follow the same general principle, but map the test token to a vector of bin logits instead. The spline variant reuses this same architecture and differs only in the training objective.

The head of the flow-based model takes as input the representation of y_{test} , the current state y_t , and the time variable t . These inputs are processed through separate embedding pathways, with t encoded via sinusoidal features. The context representation is normalized

before further processing in order to keep it on a scale comparable to the embeddings of y_t and t , thereby preventing it from dominating the subsequent computation. The resulting embeddings are then concatenated and projected into a shared hidden representation. In addition, the flow head uses modulation blocks similar in spirit to those of Peebles and Xie (2023), allowing for a more structured conditioning on context and time.

5.2 Pretraining Priors

A central design choice in PFNs is the specification of a *data prior*. As emphasized by Müller et al. (2021), this prior encodes assumptions about plausible data-generating mechanisms and thereby determines which prediction tasks the model is exposed to during pretraining. From a theoretical perspective, good PFN performance can only be expected if the prior is sufficiently rich and does not place too much mass on unfavorable regions of the task space (see Nagler, 2023). Motivated by this consideration, and building on the prior design principles discussed in Chapter 2.3, we introduce two priors of deliberately different complexity: a *simple linear prior* and a *more complex SCM-based prior*. The comparison between them is intended to probe the empirical benefit of a broader and more realistic prior.

To ensure reproducibility while still allowing controlled sources of randomness, we use three independent random seeds throughout data generation:

- **Structural seed.** Governs the *data-generating mechanism* itself, including sampled functional relationships (e.g. graph structure and edge weights), distributional parameters and noise scales, and the resulting correlation structure. This seed also determines discrete design choices such as the number of observed features.
- **Feature seed.** Controls the sampling of the observed covariates $\mathbf{X}$. Fixing this seed while varying others allows identical feature realizations to be regenerated under the same structural mechanism, enabling datasets that share covariates but differ in their target variable.
- **Aleatoric seed.** Controls the final source of irreducible randomness—the stochastic component applied at the end of the generation process (e.g. Bernoulli sampling of labels from class probabilities). Varying this seed changes the realized labels even when $\mathbf{X}$ and the structural mechanism are held fixed. Together with the feature seed, it allows us to isolate the irreducible noise component and thereby establish a theoretical upper bound on what can be learned from any given dataset instance.

This decomposition makes the degree of determinism in each dataset instance explicit and thus provides a clearer picture of aleatoric uncertainty. Concretely, two instances sharing the same feature seed but different aleatoric seeds yield identical covariate realizations $\mathbf{X}$ while differing only in the label draws Y ; this is illustrated in Figure 5.

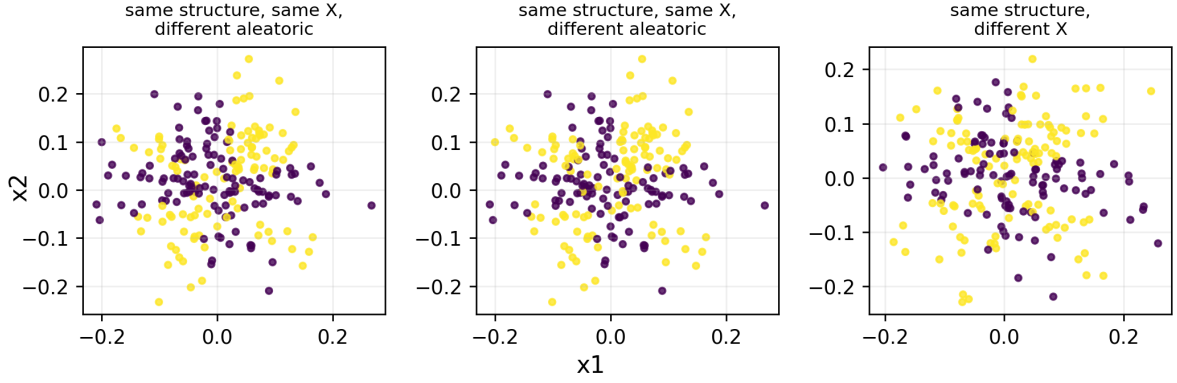


Figure 5: Three dataset instances illustrating the role of each random seed. **Left and center:** identical feature seed, hence identical covariate realizations $\mathbf{X}$, but different aleatoric seeds: labels flip for a subset of observations near the decision boundary. **Right:** same structural seed (identical underlying DGM and decision boundary) but a different feature seed, producing a fresh realization of $\mathbf{X}$.

5.2.1 Simple-prior

At a high level, this prior corresponds to a randomly generated noisy linear model. For each dataset instance, we first sample the dataset size n uniformly up to $n_{\max}$ and the feature dimension d uniformly up to $d_{\max}$. We then draw feature vectors $\mathbf{X} \in \mathbb{R}^{n \times d}$ with i.i.d. standard normal entries. A weight vector $w \in \mathbb{R}^d$ is sampled and subsequently rescaled coordinate-wise by a random log-normal factor, thereby inducing heterogeneous feature effect sizes across coordinates. In addition, a small scalar bias b is sampled to avoid overly strong class imbalance.

Targets are then generated from a noisy linear model,

$$\text{logits} = \mathbf{X}w + b + \epsilon,$$

where ϵ denotes additive Gaussian noise whose scale is chosen proportional to the average standard deviation across the columns of $\mathbf{X}$. In the binary classification setting, labels are obtained by thresholding,

$$y = \mathbf{1}(\text{logits} > 0),$$

whereas in the regression setting the logits themselves are used directly as the target. In principle, additional sparsity can be introduced by setting a subset of the coordinates of w to zero, thereby creating entirely irrelevant features; in the present implementation, however, all features remain active.

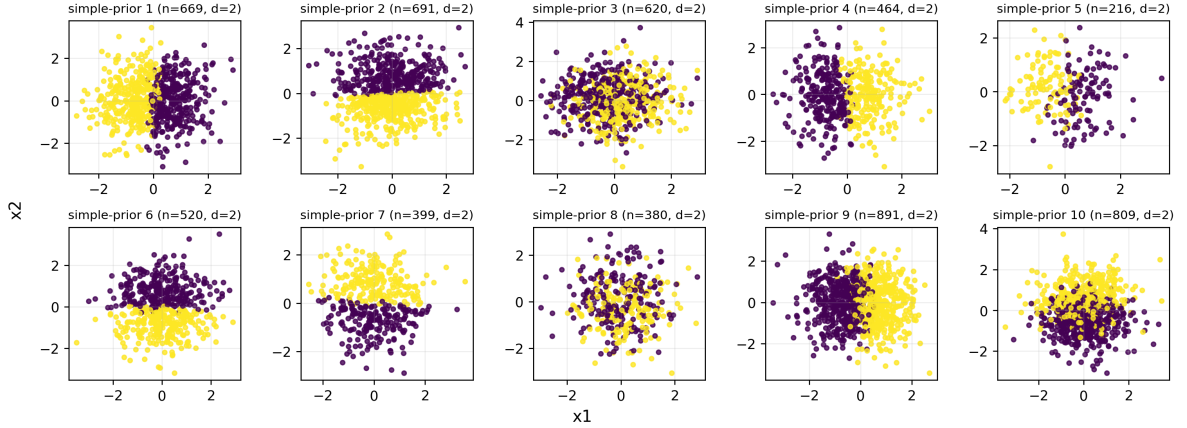


Figure 6: Ten random datasets from the simple-prior with feature dimension fixed to $d = 2$. Each task is generated by a linear decision boundary perturbed by Gaussian noise.

Overall, the simple prior defines a setting with uncorrelated input features and a linear decision boundary corrupted by controlled aleatoric noise. Most datasets generated from this prior are also rather balanced in terms of their class frequencies. One useful property of this construction is that the ground-truth weight vector w is known, which makes it possible to derive diagnostic proxies for feature relevance. In Appendix A.1, we exploit this property in two exploratory analyses: first, by relating relative feature weight to learned attention patterns, and second, by examining representations extracted from the final model layer.

5.2.2 SCM-prior

To obtain a more challenging prior, we take the SCM-inspired data-generation strategy of Hollmann et al. (2023) as a starting point. The core idea is to sample a random nonlinear DAG by constructing a random MLP: depth and hidden widths are sampled, each layer is assigned a randomly chosen activation function (power, logarithm, square root, absolute value, tanh, identity, or smooth ReLU), and connections are randomly zeroed to introduce sparsity, thereby turning the dense network into a sparse graph. A random subset of internal nodes is selected as observed features, and one node is designated as the target. To generate a dataset, root noise variables are propagated through the DAG, after which values are collected at the selected feature nodes and at the target node.

A practical limitation of this single-DAG construction is that the resulting signal-to-noise ratio is not transparent. Because noise is injected along edges, the same root initialization can give rise to substantially different observed feature values after propagation. This makes it difficult to define a clean bayes-optimal reference that separates uncertainty about the latent data-generating mechanism from irreducible label noise.

To address this issue, we separate feature generation and label generation into two stages. First, we generate the *features* by sampling a random DAG, propagating noise through it, selecting observed nodes, and adding Gaussian feature noise. These observed features are deterministic given the feature seed. Second, we sample an independent label DAG that takes the generated features $\mathbf{X}$ as input and produces a logit score z via a

nonlinear mapping,

$$z = f_{\text{label}}(\mathbf{X}), \quad (26)$$

where f_{label} denotes the forward pass through the second DAG.

Classification. To remove any implicit bias inherited from the DAG propagation, z is standardized before the sigmoid transformation, yielding $\tilde{z}$. Applying a sigmoid with temperature T then yields

$$p = \frac{1}{1 + e^{-\tilde{z}/T}}, \quad (27)$$

and labels are sampled as

$$Y \sim \text{Bernoulli}(p). \quad (28)$$

For all binary classification experiments with the SCM-prior, the temperature was fixed to $T = 0.1$. Increasing the temperature produces a noisier decision boundary, whereas decreasing it further makes the sigmoid sharper and the label generation more deterministic. Despite the standardization of z before the sigmoid transformation, this prior can still generate datasets with substantial class imbalance, because the distribution of z may remain strongly skewed. In such cases, more probability mass lies on one side of the decision boundary than on the other, which directly translates into an imbalanced class distribution.

Regression. For regression, the standardization step is omitted. Since z is already obtained through a random DAG propagation involving multiple nonlinear stages, additive bias is implicitly absorbed into the network weights. Instead, the output is scaled by a random factor and corrupted with Gaussian noise.

Figure 7 shows ten example classification datasets sampled from the SCM-prior.

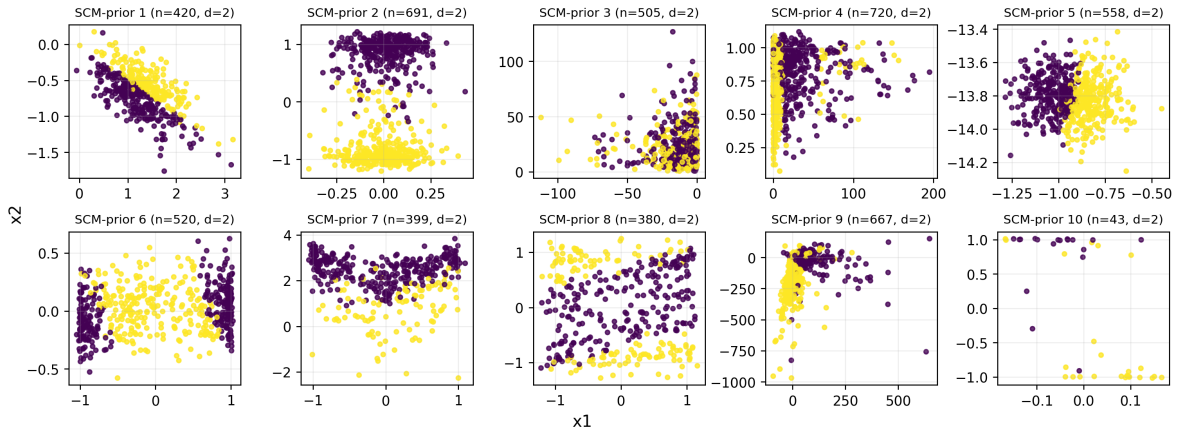


Figure 7: Ten random datasets from the SCM-prior with feature dimension fixed to $d = 2$. Each task can exhibit nonlinear, heterogeneous, periodic, or cluster-like decision structures. The SCM-prior can also output simple, linear decision boundaries, like the simple-prior.

While the introduced SCM-prior is substantially more complex than the simple-prior, it should still be regarded as only *moderately complex* when compared to the priors used in the original TabPFN-V2, and even more so in TabPFN-2.5. It is likely also less complex than the prior used for the earlier proof-of-concept TabPFN-V1 model (Hollmann et al., 2023), since that prior allows for a wider range of hyper-hyperparameter variation influencing the DAG structure, which in turn leads to deeper and wider sampled graphs.

The TabPFN-V2 prior in Hollmann et al. (2025) is richer in several additional respects. It employs a redirection-based sampling scheme and, in some cases, samples multiple disjoint subgraphs that are subsequently fused into a larger DAG (see Hollmann et al., 2025). It also uses a broader family of root noise distributions that are propagated through the graph, including uniform and mixture distributions. In addition, dependencies between features are induced explicitly by mixing features through linear combinations with mixture weights, and certain features are further distorted through Kumaraswamy warping. Most importantly, the simple- and SCM-prior considered here generate exclusively continuous features and do not include categorical variables or missing values, whereas the priors in the original TabPFN work explicitly account for both. The prior used in TabPFN-2.5 is broader still (see Grinsztajn et al., 2025), extending the diversity of data-generating mechanisms beyond the already richer TabPFN-V2 design.

5.3 Synthetic classification experiments

Hollmann et al. (2025) motivate the transition from TabPFN-V1 to TabPFN-V2 through an architectural modification that treats both the *sample dimension* and the *feature dimension* as sequence-like objects. Concretely, V2 replaces the single two-dimensional self-attention operation over the dataset with an attention scheme that alternates between row-wise and column-wise attention. This change is argued to yield better generalization when the number of samples or features at inference time differs from what was encountered during pretraining.

Beyond these conceptual arguments, however, it remains an empirical question *when* and *by how much* the V2 architecture provides a practical advantage over V1 under comparable capacity. To study this, we conduct a controlled random search over the core architectural hyperparameters for both versions, where k denotes the transformer embedding dimension, m the hidden dimension of the feedforward network, n_{layers} the number of transformer blocks, and n_{heads} the number of attention heads. Crucially, every sampled configuration is trained *twice*: once with the V1 architecture and once with the V2 architecture, while keeping $(k, m, n_{\text{layers}}, n_{\text{heads}})$ fixed within each pair. This random search is performed exclusively for the binary classification setting.

Evaluation metric on synthetic tasks. During pretraining, models are periodically evaluated on newly sampled synthetic datasets drawn from the same prior that is used for training. For each run, we report the *synthetic accuracy*, defined as the mean accuracy over the last ten evaluation checkpoints. Since these checkpoints are based on freshly sampled tasks from the pretraining distribution, this metric primarily reflects how well a model has adapted to the synthetic prior itself. At this stage, it should therefore be interpreted mainly as a practical proxy for monitoring convergence and comparing

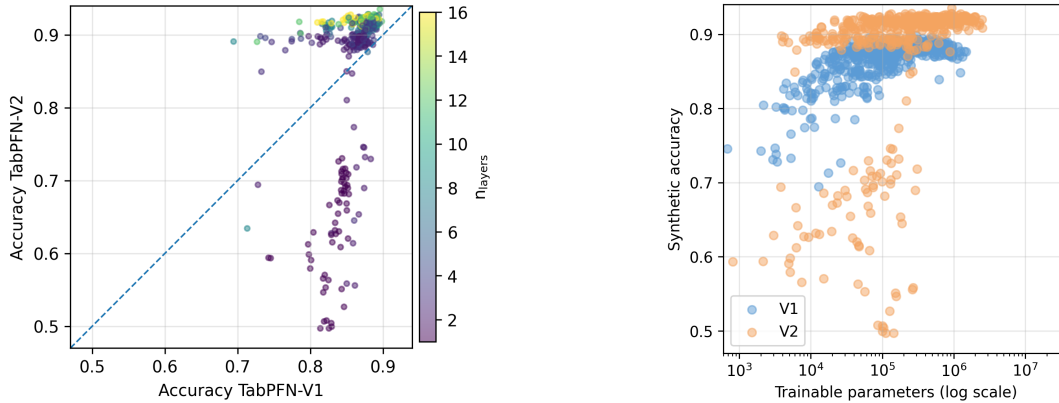
training dynamics across configurations, rather than as a direct measure of real-world generalization. The extent to which such synthetic performance transfers to real tabular datasets will be examined separately in later experiments.

Parameter counts and fairness of the comparison. Even when $(k, m, n_{\text{layers}}, n_{\text{heads}})$ are matched, TabPFN-V2 typically has more trainable parameters than TabPFN-V1. This is illustrated in the supplementary material in Appendix A.3, in particular in Figure 23. The reason is architectural: V2 applies two attention operations per layer—feature-wise and sample-wise—whereas V1 uses only a single attention mechanism over the token sequence.

At first sight, this could raise the concern that the comparison favors V2 simply because it is larger. However, the results in the main text together with the supplementary analyses suggest that this is unlikely to be the full explanation. In particular, V1 appears to reach a regime of diminishing returns relatively quickly: increasing its parameter count does not seem to yield substantial further gains in synthetic performance. This suggests that V1 would likely not benefit much from additional capacity alone. For this reason, we still regard the paired comparison as informative.

5.3.1 Classification on the simple-prior

Across the sampled configurations, the TabPFN-V2 prototype achieves higher synthetic performance than the TabPFN-V1 prototype in the majority of matched comparisons (**85.2%** of cases).



(a) Paired comparison of synthetic performance. Each point corresponds to one sampled hyperparameter configuration $(k, m, n_{\text{layers}}, n_{\text{heads}})$, trained once with TabPFN-V1 (x-axis) and once with TabPFN-V2 (y-axis). The diagonal indicates equal performance; points above it favor V2. Color encodes the depth n_{layers} .

(b) Synthetic accuracy versus the number of trainable parameters (log scale) for the simple-prior sweep. Points come in matched V1/V2 pairs that share the same $(k, m, n_{\text{layers}}, n_{\text{heads}})$ configuration (pairwise connections are shown in Appendix A.3).

Figure 8: Hyperparameters and synthetic accuracy of the prototypes for the *simple-prior* sweep.

Figure 8a provides a direct paired comparison: each point corresponds to one hyperparameter configuration, with the V1 final mean accuracy on the x-axis and the V2 final mean accuracy on the y-axis. Points above the diagonal indicate a V2 advantage. The strong concentration of points above the identity line visually indicates that V2 tends to outperform V1 in most cases. Moreover, the color coding highlights that the best-performing configurations tend to be deeper: high-accuracy points cluster among larger values of n_{layers} .

Figure 8b complements this view by plotting accuracy against the number of trainable parameters (log scale), again for matched V1/V2 runs. For this prior, V2 is most of the time better than V1 at comparable configurations. At the same time, the plot reveals a notable failure mode: a subset of V2 runs performs extremely poorly despite non-trivial capacity, indicating that performance is not a smooth monotone function of parameter count alone.

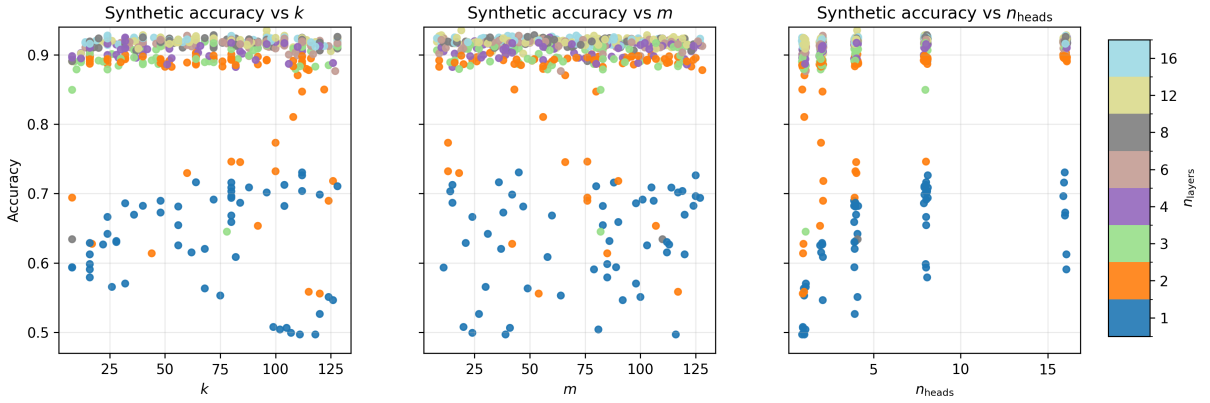


Figure 9: TabPFN-V2 prototype synthetic performance on simple-prior across key hyperparameters. Each panel shows final mean accuracy versus one hyperparameter (k , m , and n_{heads}), with a discrete color scale indicating depth n_{layers} .

To better understand these outliers, Figure 9 focuses on the V2 architecture and displays synthetic accuracy as a function of the main hyperparameters, with network depth shown via a discrete color scale. The poorly performing V2 runs are concentrated almost exclusively among *very shallow* models, most notably those with $n_{\text{layers}} = 1$ or 2. This suggests that, under the simple-prior, the V2 architecture is particularly sensitive to insufficient depth: the alternating-attention design may require a minimum number of alternating row- and column-attention blocks in order to form useful representations. By contrast, once depth is sufficiently large, performance becomes markedly more stable.

To quantify this observation and assess how predictive the hyperparameters are for model performance, we fit a surrogate model in the spirit of Breiman (2001) using gradient boosting, and applied the permutation importance procedure implemented in `scikit-learn`. The target variable was the synthetic accuracy, while the feature vector consisted of the corresponding hyperparameter configuration. Permutation importance measures the extent to which predictive performance deteriorates when the values of one feature are randomly permuted, thereby destroying its association with the target. If a feature is informative for predicting model performance, permuting it should lead to

a noticeable degradation of the surrogate model. As performance metric, we used the coefficient of determination R^2 .

The results show that n_{layers} is by far the most important hyperparameter (1.6678 ± 0.1057), indicating that the surrogate model breaks down almost completely once depth information is removed. The remaining hyperparameters have only comparatively minor effects. This provides quantitative support for the qualitative pattern observed in the plots.

Taken together, the simple-prior sweep supports two conclusions. First, when comparing matched hyperparameter configurations, V2 typically outperforms V1 on this *synthetic* prior. Second, this advantage is not uniform across the search space: V2 exhibits a pronounced *depth threshold*, below which performance can collapse.

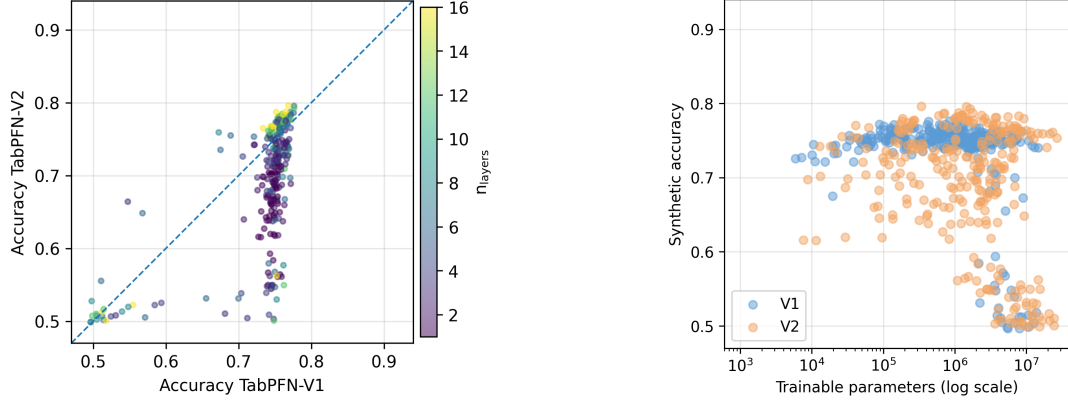
5.3.2 Classification on the SCM-prior

We repeat the same analysis for the more complex SCM-prior. Relative to the simple-prior sweep, this experiment differs in several practical respects: (i) the hyperparameter search space is larger, (ii) training is run for longer, (iii) a learning-rate schedule is used, (iv) the maximum number of samples per dataset is increased from $n_{\text{max}} = 200$ to $n_{\text{max}} = 500$, and (v) due to computational constraints, the random search comprises only 281 trials. The extended training setup is motivated by the substantially richer nonlinear dependencies present in SCM datasets, which led to noticeably slower convergence than in the simple linear setting, where performance typically stabilized much earlier. Further details on the training setups of both sweeps are provided in Appendices A.3 and A.4.

Despite these adjustments, the evaluation protocol remains unchanged. Each sampled configuration $(k, m, n_{\text{layers}}, n_{\text{heads}})$ is trained once with the TabPFN-V1 architecture and once with TabPFN-V2. The resulting *synthetic accuracy*, averaged over the final ten evaluation checkpoints on freshly generated synthetic datasets, serves as the comparison metric.

The qualitative picture for the SCM sweep differs markedly from the simple-prior results. Across the matched comparisons, TabPFN-V2 outperforms TabPFN-V1 in only about **35%** of sampled configurations. At the same time, the *best-performing* configurations overall are still V2 models, indicating that V2 can reach superior performance on the SCM-prior but only within a narrower region of the hyperparameter space, whereas V1 appears more robust across configurations.

Figure 10a mirrors the diagnostic plot used in the simple-prior sweep. As before, higher-performing runs tend to be associated with deeper models, particularly among configurations where V2 dominates. However, the SCM sweep reveals an additional failure mode: a noticeable cluster of *deep* configurations still performs poorly (visible in the lower-left area of the plot). Moreover, the dispersion is again more pronounced for V2 than for V1, reinforcing the impression that V2 is more sensitive to unfavorable architectural choices. Figure 10b complements this view by relating accuracy to the number of trainable parameters. Two patterns stand out. First, V1 accuracies form a comparatively tight band, suggesting stable performance across a wide range of capacities. Second, V2 exhibits a much more diffuse distribution, including a clearly separated low-performance cluster at high parameter counts. Notably, a corresponding degradation is also visible for V1,



(a) Paired comparison of synthetic performance on *SCM*. Each point corresponds to one sampled hyperparameter configuration $(k, m, n_{\text{layers}}, n_{\text{heads}})$, trained once with TabPFN-V1 (x-axis) and once with TabPFN-V2 (y-axis). The diagonal indicates equal performance; points above it favor V2. Color encodes n_{layers} .

(b) Synthetic accuracy versus the number of trainable parameters (log scale) for the *SCM* sweep. Points come in matched V1/V2 pairs sharing the same $(k, m, n_{\text{layers}}, n_{\text{heads}})$ configuration (pairwise connections are shown in Appendix A.4).

Figure 10: Hyperparameters and synthetic accuracy for the *SCM* sweep.

hinting at a shared underlying cause tied to architectural capacity allocation rather than version-specific behavior.

Further analysis reveals that the configurations in this low-performance cluster share a common characteristic: a large global embedding dimension k combined with very few attention heads, resulting in an unusually large per-head dimension $d_k = k/n_{\text{heads}}$ (as introduced in Chapter 2.1). In the most extreme cases, only a single attention head is present, forcing the entire embedding capacity—sometimes 400 to 500 dimensions—onto a single head. Figure 11 visualizes this by coloring each point according to d_k : the low-performance cluster is dominated by configurations with very large d_k , regardless of overall parameter count. A small number of low-performing points with small d_k are also visible; these correspond to shallow architectures whose poor performance is attributable to insufficient depth rather than head capacity.

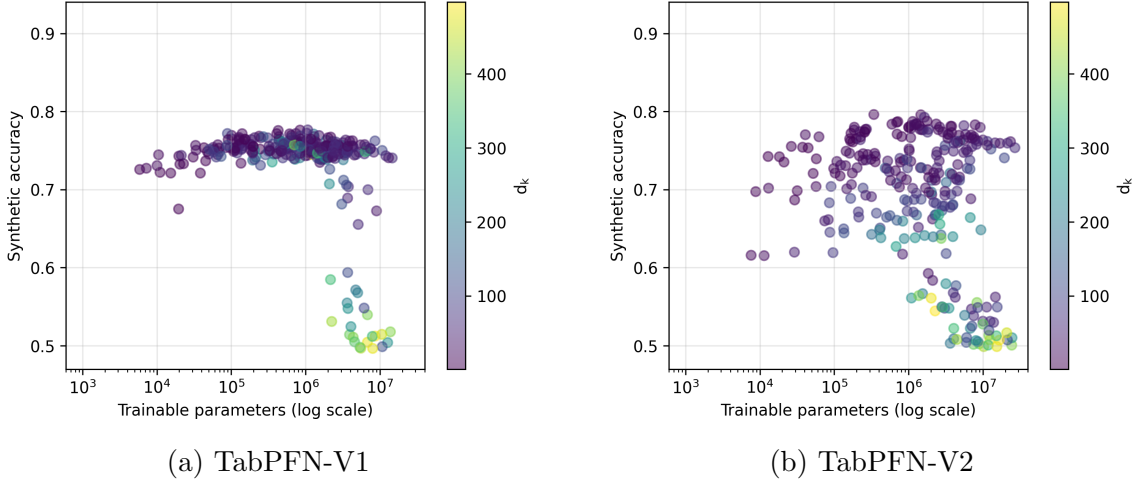


Figure 11: Synthetic accuracy versus trainable parameters (log scale) for the *SCM* sweep, colored by the per-head dimension $d_k = k/n_{\text{heads}}$ (lighter colors indicate larger d_k). The pronounced low-performance cluster is dominated by configurations with very large d_k , suggesting that allocating too much capacity per head is detrimental under this prior.

This finding is consistent with the original transformer work of Vaswani et al. (2017), who observed that single-head attention performs worse than multi-head attention at the same global embedding dimension, but that too many heads with very small d_k can also be detrimental. The intuition is that multiple heads allow the model to jointly attend to information from different representation subspaces. Voita et al. (2019) provide direct empirical support for this in encoder-based language models, finding that different attention heads specialize functionally: some attend to adjacent positions, others capture syntactic relations, and others respond to rare tokens. This division of labor suggests that collapsing all capacity into a single head prevents the model from learning such complementary representations. Although TabPFN shares a similar transformer architecture, it would be interesting to investigate whether analogous specialization emerges across attention heads in the PFN setting—a direction we leave for future work. Remarkably, we did observe trials in which synthetic performance was competitive even with d_k as small as 1 or 2.

Not surprisingly, applying the same surrogate-model analysis as before to the *SCM* sweep yields a different ranking of hyperparameter importance. Here, the per-head dimension d_k is the dominant variable (permutation importance 0.7617), followed by the global embedding dimension k (0.3014) and depth n_{layers} (0.1678). By contrast, n_{heads} contributes little once d_k is taken into account. This quantitatively supports the visual pattern in Figure 11: under the *SCM* prior, performance depends primarily on how embedding capacity is distributed across attention heads, rather than merely on the number of heads itself.

5.3.3 Synthetic performance and generalization

The previous experiments showed that synthetic performance is highly sensitive to architectural choices, particularly for the V2 architecture. Under the simple-prior, insufficient depth can lead to severe degradation, whereas under the *SCM*-prior, performance depends

strongly on how embedding capacity is distributed across attention heads. This naturally raises the question of how informative synthetic performance actually is with respect to downstream real-world performance.

To investigate this, we select from each synthetic sweep the best-performing V2 model, where *best-performing* refers specifically to the model that attains the highest *synthetic accuracy within that sweep*. We refer to the model that performs best in the *simple-prior sweep*—and is also pretrained on the *simple prior*—as *simple-native*. Analogously, the model that performs best in the *SCM-prior sweep*—and is pretrained on the *SCM prior*—is referred to as *SCM-native*. Thus, the term *native* indicates both the pretraining prior and the fact that the corresponding hyperparameter configuration was selected as optimal *within that same prior regime*. Since the simple-prior sweep used shorter training runs, we retrain both selected models under a common training protocol. We then monitor, throughout training, not only their synthetic performance but also their performance on 17 real-world datasets from TabArena (see Erickson et al., 2025), a continuously maintained benchmark for tabular machine learning.

All models are trained on $n_{\text{datasets}} = 100,000$ synthetic datasets, each containing up to 500 samples and 20 features, i.e. with $n_{\text{max}} = 500$ and $d_{\text{max}} = 20$. At every evaluation checkpoint, performance on real data is averaged across all 17 benchmark datasets, using 10 repeated evaluations per dataset, resulting in 170 real-data evaluations per checkpoint. The resulting training trajectories are shown in Figure 12.

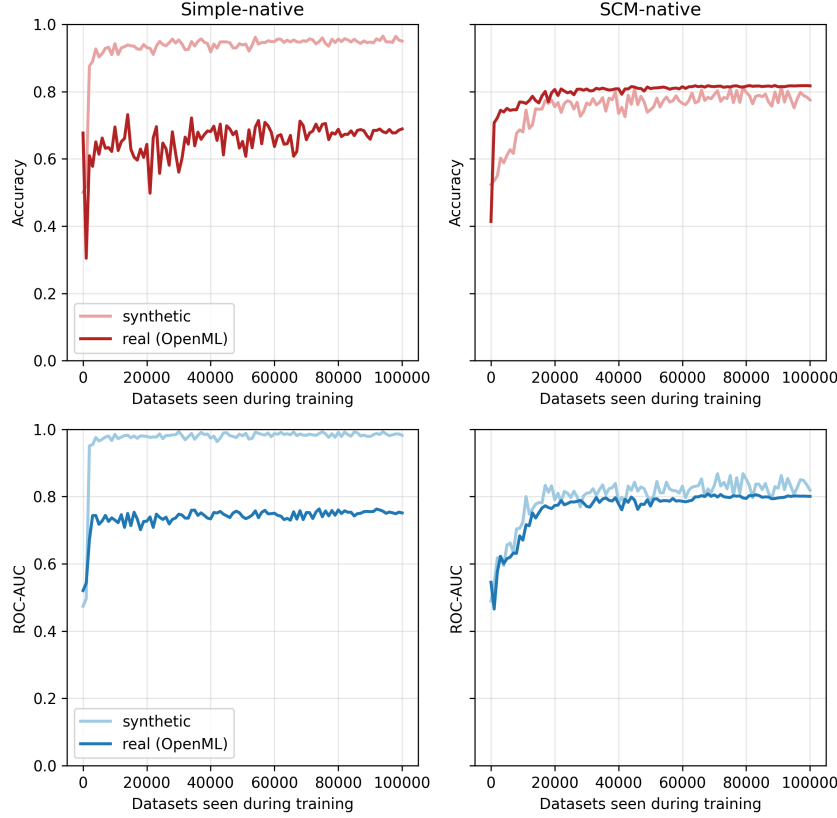


Figure 12: Training trajectories of synthetic and real-world performance for the simple-native and SCM-native TabPFN-V2 models. The top row shows accuracy, the bottom row ROC-AUC. The left column corresponds to the simple-native model, and the right column to the SCM-native model.

Several patterns stand out. For the simple-native model (left column), there remains a pronounced gap between synthetic and real-world performance, both in accuracy and in ROC-AUC. Although the model converges well on the synthetic prior, this improvement translates only partially to downstream datasets. This suggests that the simple-prior induces a systematic mismatch: even after optimization has largely converged, a non-negligible approximation error remains when transferring to real-world data.

The SCM-native model exhibits a different pattern. Its synthetic performance is shifted downward relative to the simple-native case, indicating that the underlying synthetic tasks are harder to fit. At the same time, its real-world ROC-AUC improves with a noticeable delay, but eventually reaches a higher level than in the simple-native setting. In other words, the more challenging prior appears to provide a more informative training signal for downstream real-world performance. A particularly striking observation is that, for the SCM-native model, real-world accuracy even exceeds synthetic accuracy. A plausible explanation is that the model benefits from exploiting stronger majority-class prediction behavior on real datasets, which are often more imbalanced than the corresponding synthetic tasks. In contrast, the simple-native model shows a less stable real-world accuracy curve, especially in the earlier stages of training. One possible interpretation is that, early in training, the model has not yet learned to deal robustly

with class imbalance, which would primarily affect threshold-based accuracy rather than ranking-based ROC-AUC. The SCM-native model, by comparison, appears to acquire a more stable decision behavior much earlier.

Overall, these results suggest that synthetic performance should be interpreted only as a limited proxy for downstream performance, and only under the condition that the underlying prior is sufficiently realistic. When the prior is misspecified, convergence on synthetic tasks does not guarantee proportional gains on real data, and a persistent gap between synthetic and downstream performance may remain. For a more fine-grained, dataset-level view of these dynamics, Appendix A.5 reports the corresponding training trajectories for each benchmark dataset individually.

5.4 Benchmarking

We now turn to a more fine-grained analysis of real-world performance at the dataset level. This section continues the line of argument from the previous subsection: We begin with a comparison of ROC-AUC for the prior-native models, that is, the configurations that achieved the highest synthetic accuracy within their respective sweeps.

5.4.1 Binary classification benchmarking

In both sweeps, the best-performing configuration corresponds to the V2 architecture. As in the preceding experiments, all models are pretrained on $n_{\text{datasets}} = 100,000$ synthetic datasets with $n_{\text{max}} = 500$ and $d_{\text{max}} = 20$. For completeness, we additionally evaluate V1 under the same selected hyperparameter configurations. As a reference baseline, we also include the publicly available default `TabPFNClassifier` from the `tabpfn` package, corresponding to the V2 model of Hollmann et al. (2025). For this model no further training takes place.

Before evaluation, all datasets are processed using a uniform preprocessing pipeline. Numerical features are median-imputed, while categorical features are first imputed with their most frequent value and subsequently ordinal-encoded. In addition, the model-internal preprocessing described in Chapter 5.1 is applied. Although Erickson et al. (2025) propose a standardized evaluation protocol, TabArena serves here only as a source of benchmark datasets. The reason is that, due to computational constraints, we limit the sequence length to 2000. As a consequence, all models—including the TabPFN baseline—must be evaluated within our own setup. We therefore use 50 repeated holdout splits, each consisting of 90% training data (context data in the PFN setting) and 10% evaluation data (query data). Since the V1 prototype has a fixed maximum feature embedding size (here, 20 features), it is omitted on datasets for which this limit is exceeded.

Figure 13 reports ROC-AUC results for the simple-native model. Overall, the simple-native models perform below the baseline on most datasets. The largest gaps between the V1/V2 prototypes and the baseline are observed for `Is-this-a-good-customer`, `seismic-bumps`, and `Bank_Customer_Churn`. Comparing the two prototype versions, there appears to be little difference between V1 and V2 on most datasets. This suggests that, under such a simple pretraining prior, the additional capacity and architectural refinements of V2—in particular alternating-attention—might not be exploited effectively.

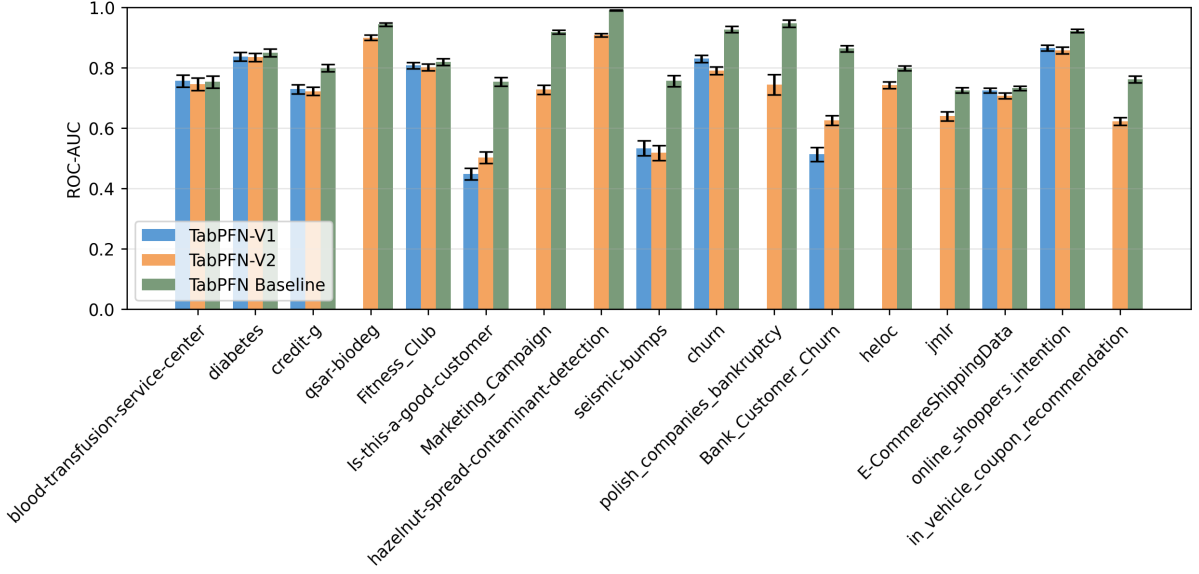


Figure 13: ROC-AUC (mean $\pm$ 95% confidence intervals over 50 repeated holdout splits) for the simple-native model.

Figure 14 shows the corresponding results for the SCM-native model. Here, performance is substantially more competitive, particularly for the V2 variant. Relative to the simple-native setting, AUC improves on most datasets, with the largest gains occurring precisely on those datasets where the simple-native models performed worst. In this setting, V2 appears to outperform V1 on most datasets in terms of ROC-AUC, suggesting that the additional capacity and architectural refinements can now be utilized more effectively.

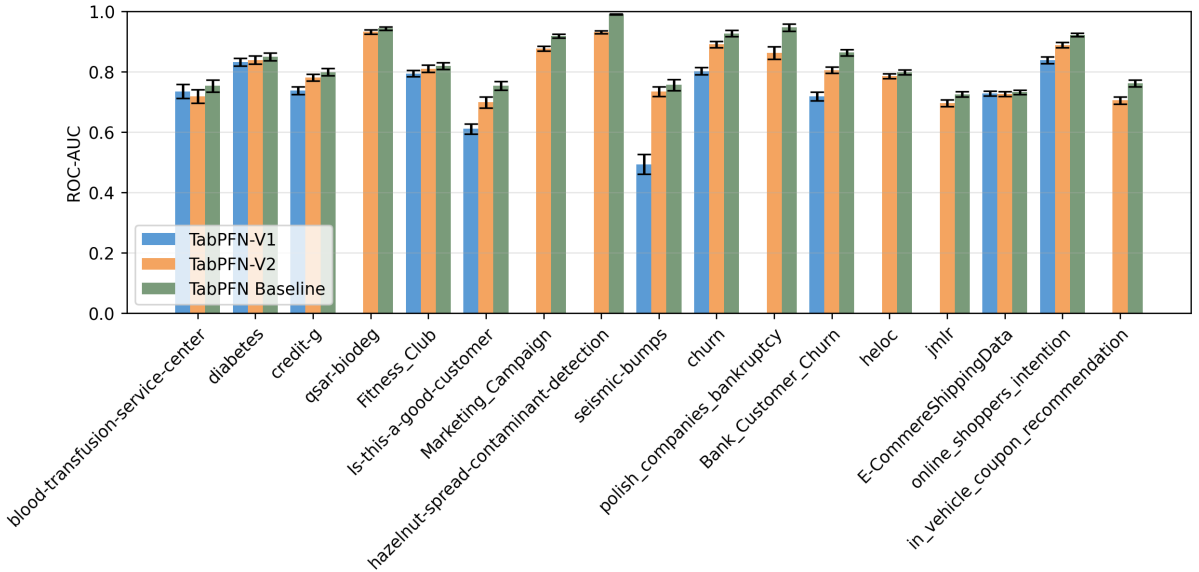


Figure 14: ROC-AUC (mean $\pm$ 95% confidence intervals over 50 repeated holdout splits) for the SCM-native model.

These results already provide strong evidence that the choice of pretraining prior has a substantial effect on downstream performance. It is important to note, however, that the two models compared so far—simple-native and SCM-native—differ not only in their pretraining prior but also in their architectural configuration; the corresponding hyperparameter settings are summarized in Table 2. The simple-native model uses a larger embedding dimension k with fewer layers, whereas the SCM-native model is deeper and uses a smaller k . This is a direct consequence of the sweep design: configurations were selected according to synthetic performance under their respective priors, and a large k proved detrimental in the SCM sweep. As a result, a direct comparison between the two native models confounds the effect of the pretraining prior with that of the architectural configuration.

To make the terminology precise, we distinguish between *simple-optimal* and *SCM-optimal* as names for the hyperparameter configurations that performed best within the simple-prior and SCM-prior sweeps, respectively. The terms *simple-native* and *SCM-native*, by contrast, refer to the corresponding full models in which these configurations are pretrained on their matching prior. In other words, *native* denotes the matched combination of optimal configuration and pretraining prior. This distinction motivates the factorial analysis presented in the following.

Table 2: Hyperparameter configurations of the evaluated models. Baseline is the configuration with similar capacity as in Hollmann et al. (2025). The prototypes were trained on 100,000 datasets, each with up to 500 samples and up to 20 features.

Configuration	k	m	n_{layers}	n_{heads}	Parameters
simple-optimal	96	19	12	4	939,590
SCM-optimal	40	101	16	4	340,778
Baseline (V2)	192	768	12	6	7,113,410

To disentangle the effects of architecture and pretraining prior, we train two additional controlled variants by crossing the two selected architectural configurations with the two priors, yielding four combinations in total:

- the simple-optimal configuration pretrained on the simple prior (*simple-native*),
- the SCM-optimal configuration pretrained on the SCM prior (*SCM-native*),
- the simple-optimal configuration pretrained on the SCM prior,
- the SCM-optimal configuration pretrained on the simple prior.

This factorial design makes it possible to attribute downstream performance differences separately to (i) the choice of pretraining prior and (ii) the architectural configuration. Figure 15 illustrates this setup: moving vertically along the blue arrows isolates the effect of the prior by holding the configuration fixed while changing the pretraining prior; moving horizontally along the orange arrows isolates the effect of the architectural configuration by holding the prior fixed while changing the hyperparameter configuration.

Since the performance differences were more pronounced for the V2 architecture than for V1, we restrict the subsequent analysis to V2. Because this experiment enables a controlled counterfactual comparison, we refer to these adjustments as *treatments*.

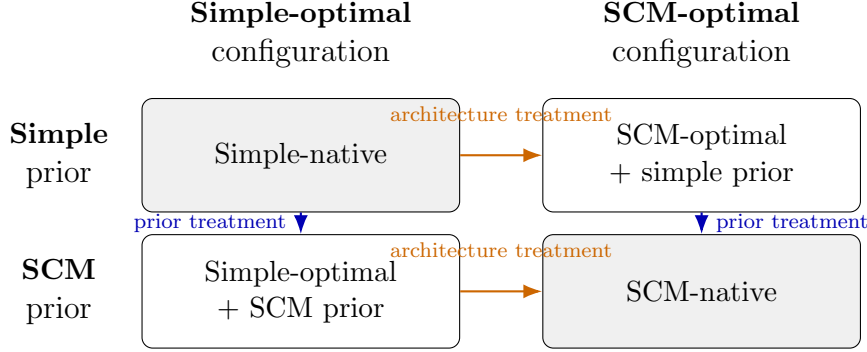


Figure 15: Factorial design used to separate configuration effects from pretraining-prior effects. Columns correspond to the two hyperparameter configurations selected as optimal in the simple-prior and SCM-prior sweeps, respectively. Rows correspond to the pretraining prior. Shaded cells indicate the two native models, i.e. the matched combinations of optimal configuration and corresponding prior.

Figure 16 presents the results of the factorial evaluation. The two panels in the top row show the prior treatment (i): the left panel holds the simple-optimal configuration fixed and switches the pretraining prior from simple to SCM; the right panel repeats this for the SCM-optimal configuration. In both panels, most points shift noticeably to the right, indicating higher AUC values under the SCM-prior. This pattern is consistent across the majority of datasets, suggesting a clear performance gain from the more expressive prior. For some datasets, the effect is less pronounced, and overlapping confidence intervals suggest that the differences may not be statistically significant.

The middle row shows the architecture treatment (ii). The effect is more modest: replacing the simple-native configuration with the deeper SCM-native architecture yields small but consistent improvements across datasets under both priors. While the gains are smaller than those from the prior treatment, the SCM-native model achieves the strongest overall performance when also trained on the SCM-prior, suggesting that architecture and prior interact and are best matched in complexity.

The bottom panel reveals a striking failure mode. When the simple prior is held fixed but model capacity is increased to the scale of the publicly available TabPFN baseline (last row in Table 2), performance degrades substantially across some datasets. This could indicate overfitting: a large-capacity model trained on a simple, low-diversity prior fails to generalize, as the model has sufficient capacity to memorize the narrow training distribution rather than learning transferable patterns.

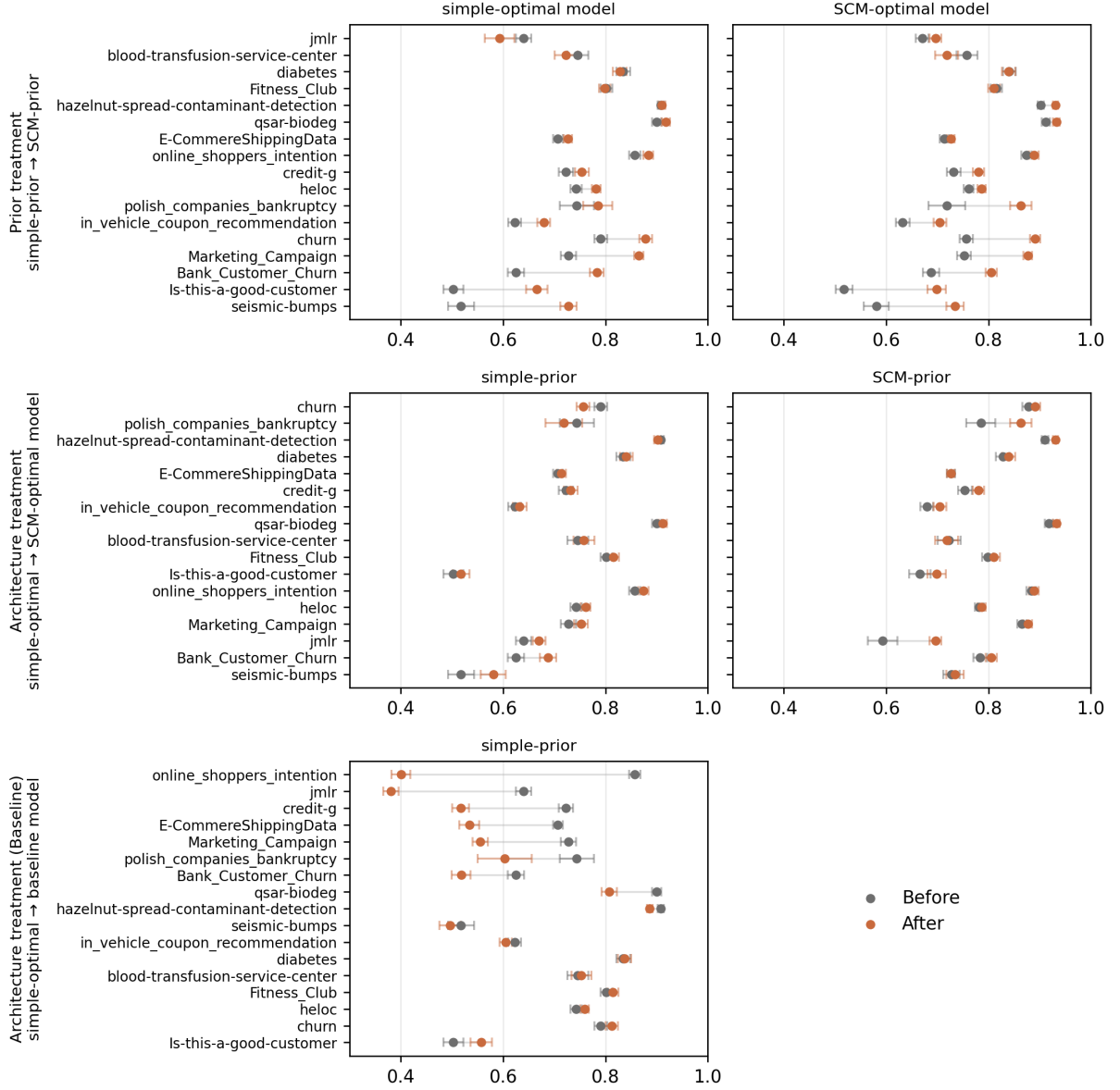


Figure 16: Treatment effects on real-world AUC. Each row shows a different treatment; each panel displays per-dataset performance before and after the treatment. Top row: prior treatment, replacing the simple prior with the SCM prior for each configuration. Middle row: architecture treatment, replacing the simple-optimal configuration with the SCM-optimal configuration under each prior. Bottom: replacing the simple-optimal configuration with the larger baseline model under the simple prior.

ROC-AUC captures the ranking behavior of the model and is therefore insensitive to the decision threshold itself. In addition to ranking, however, the pretraining prior also appears to influence how readily the model departs from approximately balanced decisions. A first indication of this already emerged in Section 5.3.3: the SCM-native model reached a stable and comparatively high accuracy much earlier during training, whereas the simple-native model remained unstable for longer and converged to a lower final level. This is consistent with the construction of the priors themselves, since the

SCM-prior generates substantially more imbalanced datasets than the simple-prior. It is therefore natural to ask how strong this effect is and how it manifests in the model’s threshold-dependent prediction behavior on real-world datasets.

Figure 17 shows how this difference in pretraining data translates into different prediction behavior when the architecture is held fixed. The x-axis reports the constant prediction ratio of the baseline model (the original TabPFN-V2), while the y-axis shows the corresponding constant prediction ratio of the prototype TabPFN-V2. Here, the constant prediction ratio is defined as the fraction of test points assigned to the more frequently predicted class, i.e. the share of the majority prediction on the test set. Color indicates the overall class imbalance of the dataset, measured as the fraction of the more dominant class. Each arrow corresponds to one dataset and connects the two prior conditions: it starts at the result obtained after pretraining on the simple-prior and ends at the result obtained after pretraining on the SCM-prior. Thus, analogous to Figure 16, the figure visualizes the prior treatment effect, though now with respect to constant-prediction behavior rather than ROC-AUC. Across both architectures (simple-optimal on the left and SCM-optimal on the right), switching from the simple-prior to the SCM-prior tends to move points closer to the diagonal for most datasets. This indicates that the prototype’s tendency to make constant, majority-class predictions becomes more similar to that of the original TabPFN-V2 baseline. This effect is more consistent for the SCM-optimal architecture, where the vast majority of datasets shift toward the diagonal. In contrast, for the simple-optimal architecture, some datasets instead shift upward, increasing the constant prediction ratio—even in cases where it was already too high before the prior-treatment. Overall, this suggests that the SCM-optimal model adapts its prediction behavior more appropriately to dataset imbalance.

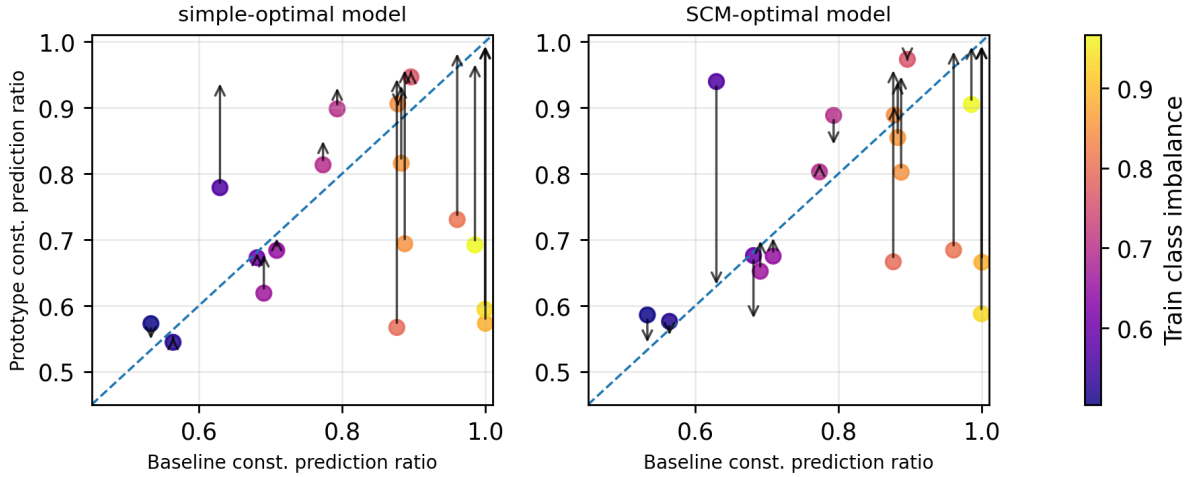


Figure 17: Effect of the prior treatment on constant-prediction behavior. Each arrow connects the result obtained after pretraining on the simple-prior to the result obtained after pretraining on the SCM-prior, while keeping the architectural configuration fixed. The x-axis shows the constant prediction ratio of the baseline TabPFN-V2 model, and the y-axis the corresponding ratio of the prototype model. Color indicates dataset imbalance.

5.4.2 Regression benchmarking

The previous chapter showed that, in the binary classification setting, pretraining on the SCM-prior generally leads to stronger generalization performance than pretraining on the simple-prior—both in terms of AUC but also threshold calibration. Since this improvement appears to stem from a closer match between the data prior and the downstream data distribution, it is reasonable to expect the SCM-prior to be preferable in the regression setting as well. We therefore restrict the subsequent analysis to models pretrained on the SCM-prior. As in the classification benchmark, pretraining is carried out with $n_{\max} = 500$ and $d_{\max} = 20$, but the total number of synthetic training datasets is increased to $n_{\text{datasets}} = 1.5$ million. Again, we use Erickson et al. (2025) as a dataset source, including 12 regression datasets.

As in Erickson et al. (2025), we report RMSE for the regression datasets. In addition, we include the negative log-density (equivalently, the negative log-likelihood, NLL), which can be interpreted as a distributional loss. For the flow-based model, the predictive density is available only implicitly through samples. RMSE can nevertheless be computed by averaging over these samples to obtain the predictive mean. For the NLL, we estimate pointwise log-densities using kernel density estimation (KDE) with Silverman’s rule-of-thumb bandwidth selection (Sheather, 2004). Analogously to the classification benchmark, we include the publicly available **TabPFNRegressor** from the **tabpfn** package as a reference baseline. We use the same benchmarking protocol as in the classification setting, namely repeated holdout over 50 evaluations with a 90/10 train–test split.

To enable meaningful comparison across datasets with different scales, we evaluate both metrics relative to the baseline on a per-trial basis. For RMSE, we compute the ratio between the RMSE of each model and the RMSE of the baseline model for every evaluations. For NLL, we compute the difference between the model NLL and the baseline NLL for each evaluations, translating into a logarithmic likelihood ratio. These paired quantities are then averaged across trials, and standard errors as well as corresponding confidence intervals are computed.

All regression models were trained with the shared hyperparameter configuration

$$k = 120, \quad m = 128, \quad n_{\text{layers}} = 12, \quad n_{\text{heads}} = 6.$$

Figure 18 reports the relative RMSE for each dataset. Both the flow-based model and the spline model exhibit performance that is competitive with the baseline on most datasets, although the baseline remains superior in the majority of cases. This can be seen by the fact that almost all models show an average relative RMSE above 1. The Riemann approach appears considerably less stable and is generally outperformed by the other models. This behavior is particularly pronounced on the **diamonds** dataset, where its deviation from the remaining models is especially large. Furthermore, for **airfoil_self_noise**, all prototype models perform substantially worse than the baseline.

Figure 19 reports the difference in NLL (ΔNLL) compared to the baseline. Overall, all prototype models exhibit broadly comparable behavior; however, their average ΔNLL values are consistently above zero, indicating that they underperform the baseline on average across datasets. A first noticeable deviation appears on the dataset **wine_quality**, where the flow-based model yields a markedly higher average ΔNLL than the other approaches. This effect should be interpreted with some caution, as several trials for the baseline

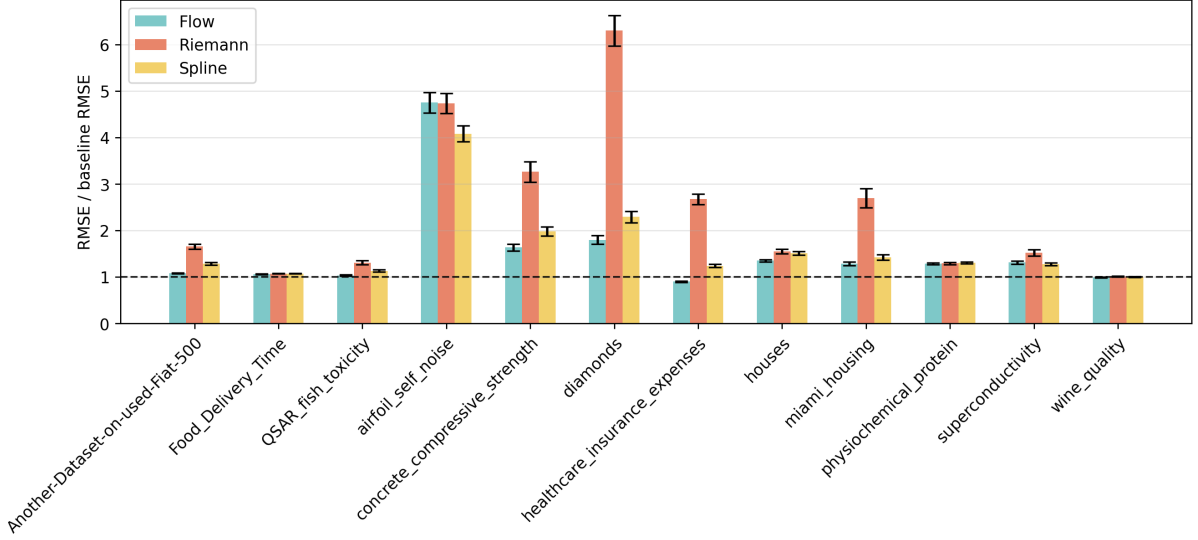


Figure 18: Relative root mean squared error on real-world regression datasets for the flow-based, Riemann, spline, and TabPFN-V2 baseline models. Bars show the mean ratio $\text{RMSE}/\text{RMSE}_{\text{baseline}}$ across repeated trials, and error bars indicate 95% confidence intervals. Values below 1 indicate improvement over the baseline, whereas values above 1 indicate worse performance.

model produced $-\infty$ values and were therefore excluded, which would otherwise have led to substantially worse relative performance of the baseline. A more extreme anomaly occurs for the flow-based model on the dataset `healthcare_insurance_expenses`. In this case, ΔNLL is substantially larger than for the other models and exhibits exceptionally high variance (not displayed in the plot) across runs, even though the RMSE remains competitive. For this dataset, the y-axis is truncated to avoid severe distortion of the visualization.

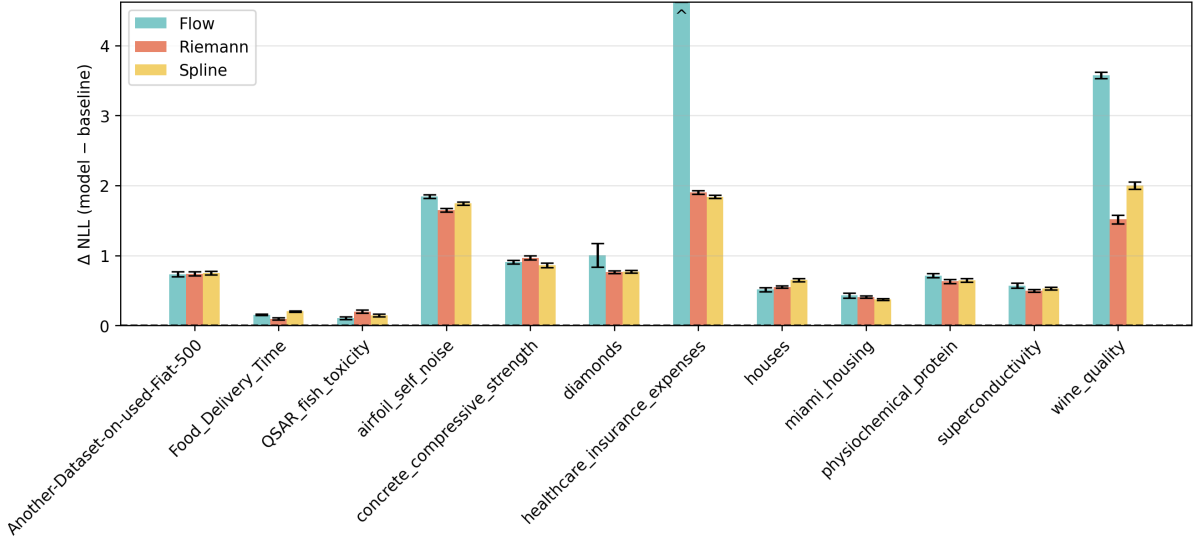


Figure 19: Difference in negative log-likelihood (NLL) relative to the TabPFN-V2 baseline on real-world regression datasets for the flow-based, Riemann, and spline models. Bars show the mean $\Delta\text{NLL} = \text{NLL}_{\text{model}} - \text{NLL}_{\text{baseline}}$ across repeated trials, and error bars indicate 95% confidence intervals. Values below 0 indicate improvement over the baseline, whereas values above 0 indicate worse performance. For the `wine_quality` dataset, the effective number of evaluations is reduced, as trials with $-\infty$ values, produced by the baseline, were excluded.

A closer inspection reveals that the poor average NLL of the flow-based model for `healthcare_insurance_expenses` is not caused by uniformly bad predictions. For most test points, the flow model assigns reasonable probability mass and achieves NLL in a competitive range. However, a small number of test points lead to extremely low log-densities. On these points, the model not only misses the target by a considerable margin, but also assigns an overly narrow predictive distribution. As a result, the observed target lies in the extreme tail of the predicted distribution, which causes the NLL to deteriorate drastically. In other words, for this dataset, the model sometimes is overly confident precisely where it is wrong. This behaviour suggests that the flow-based regression head can be unstable on certain real-world datasets, especially in regions where uncertainty should be larger.

Since TabPFN-style models produce full predictive distributions, we additionally assess distributional quality using the 1-Wasserstein distance, also known as the Earth Mover’s distance (see Gabriel and Marco, 2019), between the predicted PPD $p_{\theta}(y | x, D)$ and the ground-truth conditional distribution $p(y | x)$. It is important to note that these two distributions do not generally coincide in the finite-data setting: the PPD reflects uncertainty induced by the limited context D and can therefore be broader than the underlying data-generating distribution. Consequently, the Wasserstein distance should be interpreted primarily as a relative measure of distributional fidelity across models evaluated under identical information constraints, rather than as an absolute measure of model correctness.

The 1-Wasserstein distance can be computed entirely from samples and is therefore

particularly well suited to the flow-based model. Since the regression target is one-dimensional, we consider the case $c = 1$; see the notation introduced in Chapter 2.4. Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ denote two samples of equal size drawn from two distributions, and assume they are sorted in ascending order, i.e.,

$$x_1 \leq x_2 \leq \cdots \leq x_n \quad \text{and} \quad y_1 \leq y_2 \leq \cdots \leq y_n.$$

Then the 1-Wasserstein distance is given by

$$W_1(\mathbf{x}, \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n |x_i - y_i|, \quad (29)$$

which, in one dimension, reduces to the ℓ_1 distance between the sorted sample vectors (Gabriel and Marco, 2019).

Because this evaluation is entirely sample-based, it also requires samples from the ground-truth conditional distribution. This is only possible in a synthetic setting, and we therefore return to synthetic regression data for this experiment. Concretely, we generate data from two different function classes, *linear* and *wiggly*, and combine each of them with two noise distributions, *Gaussian* and *gamma-skewed*, yielding four synthetic configurations in total. The wiggly setting is constructed as a random combination of trigonometric functions and thus introduces functional structure that was not explicitly present during pretraining.

Figure 20 reports the resulting Wasserstein distances. A striking pattern emerges in the linear settings: the bucket-based models, *Riemann* and *Spline*, exhibit substantially worse distances than the other approaches. Inspection of the corresponding predictive distributions suggests that this behavior is caused by occasional degenerate outputs. In these cases, one side of the distribution appears reasonably shaped, while the other spreads into an excessively wide tail. This phenomenon is particularly pronounced for the Riemann model, whose problematic side can become almost uniform. The Spline model exhibits a related but less severe failure mode: instead of an approximately uniform tail, its density typically decays toward the extremes, but still remains overly diffuse. By contrast, the flow-based model achieves competitive performance in the linear settings. In the wiggly settings, all models perform more similarly overall. The Riemann model still tends to produce the weakest results, whereas the flow-based model performs best among the prototype models.

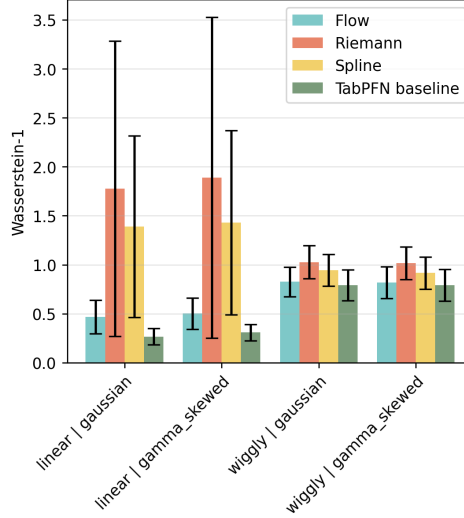


Figure 20: Wasserstein-1 distance between the learned predictive distributions and the ground-truth synthetic conditional distributions across combinations of underlying function class and noise type. Bars show mean W_1 over repeated trials, and error bars indicate 95% confidence intervals. Lower values indicate a closer match to the true predictive distribution.

6 Discussion and Limitations

The experiments point to a fairly clear overall picture. Competitive TabPFN-style prototypes can already be obtained with substantially fewer computational resources and considerably simpler priors than in the original TabPFN training setup. In particular, the results highlight that reproducing the full scale of the original pipeline is not necessary in order to achieve meaningful downstream performance. Whereas the original TabPFN-V2 was trained for roughly two weeks on approximately 130 million synthetic datasets, the classification benchmark-models in this thesis were trained on only 100,000 datasets and the regression benchmark-models on 1.5 million datasets. A similar observation holds for prior design. The results show that the pretraining prior need not be maximally complex in order to be effective. As discussed before, the SCM-prior used in the present experiments is still markedly simpler than the prior used for the original TabPFN models. At the same time, however, the pretraining prior cannot be chosen arbitrarily simple. The large performance gap between the simple-prior and the SCM-prior indicates that prior realism matters for generalization, while the comparatively smaller gap between the SCM-based prototypes and the original baseline suggests diminishing returns once the prior already captures a sufficiently broad range of relevant data-generating mechanisms. Moreover, the results imply that prior complexity and model complexity must be aligned: increasing architectural capacity alone does not appear to help when the pretraining prior is too simple for that capacity to be used effectively, and in our experiments can even harm downstream generalization. This interpretation is consistent with the theoretical perspective of Nagler (2023). In particular, Nagler argues that if the prior contains only simple models, then the corresponding optimal PFN approximation will also tend to rep-

resent only relatively simple functions. Conversely, a simple model class may not be able to benefit much from an overly complex prior. His conclusion is that good performance across diverse tasks requires sufficient capacity in both the predictor class and the prior. From this perspective, the present empirical findings fit a coherent pattern: strong PFN performance does not arise from maximizing either prior complexity or model size in isolation, but from matching the richness of the prior to the expressive capacity of the model.

The comparison between TabPFN-V1 and TabPFN-V2 also helps explain why the original TabPFN line moved from a single two-dimensional attention mechanism to an alternating-attention scheme. Across the synthetic sweeps, the strongest configurations were consistently obtained with V2, even though V2 also proved more sensitive to unfavorable hyperparameter choices. The same pattern appears in the classification benchmark: under the more suitable SCM-prior, V2 generally outperforms V1 on real-world datasets. Its advantage, however, is not purely one of predictive performance. By treating the feature dimension, like the sample dimension, as a sequence axis, V2 no longer depends on a fixed maximum feature embedding size and avoids the padding-related constraints present in V1. At the same time, this architectural change is not cost-free: for the same nominal hyperparameter configuration, V2 contains more trainable parameters than V1 and also incurs higher computational cost due to the more expensive attention operations.

The regression experiments evaluated two novel approaches for refining or even replacing the discretized PPD approach. The spline-based model appears particularly promising, as it preserves the overall architecture of the discretized regression approach and requires only comparatively minor modifications to the training objective, while improving the continuity and empirical quality of the modeled predictive distribution. It can therefore be viewed as a relatively low-cost extension of the existing framework, especially when operating with a coarser discretization of the PPD, rather than using the 5000 bins employed in the original work. The flow-based model is more ambitious: it replaces the conventional regression head with a fully generative mechanism and thus represents a conceptually different approach to predictive posterior modeling. Its results are competitive overall, especially in terms of RMSE and Wasserstein distance, but the NLL analysis also shows that its predictive uncertainty is not yet consistently calibrated. This underscores that the flow-based approach still requires further architectural refinement and more extensive tuning before it can be considered a robust alternative to more established designs. The flow-based approach could also be of interest in data privacy settings, where generative models can be used to produce synthetic data in the presence of sensitive information. However, the current approach is limited to sampling from the PPD and does not generate feature values or their dependencies, which would need to be addressed for such applications.

Despite these encouraging results, several limitations should be kept in mind. First, the architectural comparison between V1 and V2 was carried out only for binary classification. It therefore remains unclear to what extent the same conclusions extend to multiclass classification or to regression. Multiclass classification in particular was left outside the scope of this thesis, mainly because a sufficiently thorough analysis would

have required substantially more computational resources. Second, the experiments covered only a restricted range of dataset regimes during pretraining. In particular, $n_{\max}$ and $d_{\max}$ were chosen to be relatively small compared to the original TabPFN models, and only limited variation of these quantities was explored. Although the benchmark results indicate that the models can generalize beyond these training limits to some extent, this question was not studied systematically. Third, the optimization setup was not investigated exhaustively. While the SCM-prior sweep employed linear warmup and cosine decay, and the simple-prior sweep used a simpler training setup, learning-rate schedules themselves were not varied systematically. More generally, the sweeps focus primarily on architectural hyperparameters and do not attempt to optimize the full training pipeline. Fourth, the real-world benchmarking protocol was necessarily simplified relative to the original TabArena setting. In both classification and regression, the dataset size was capped at $n = 2000$ for computational reasons, even though several benchmark datasets are larger than that. It is therefore plausible that additional differences between models would emerge in larger-context regimes, especially since the original TabPFN models were trained on larger datasets and are designed to benefit from such settings. Fifth, the benchmarking baseline was restricted to TabPFN-V2 rather than the more recent TabPFN-2.5 model. This choice was made for practical and conceptual reasons: TabPFN-V2 is more readily accessible, and it provides a clearer and fairer point of comparison for the prototype models developed in this thesis. In particular, the goal was not to compete with the most advanced production-ready model, but to evaluate the proposed ideas against a well-established reference architecture corresponding to the first large-scale TabPFN model (Hollmann et al., 2025) following the initial proof-of-concept (Hollmann et al., 2023). Nonetheless, this also implies that the reported results do not reflect the current state of the art, and future work should include comparisons to more recent model versions.

A further practical limitation is that the implementations developed in this thesis should be understood as research prototypes rather than deployment-ready systems. While they are sufficient to support the empirical comparisons carried out here, they were not optimized for computational efficiency, numerical robustness, or production use. This is not merely an engineering detail. Cleaner and more efficient implementations would likely have enabled broader hyperparameter searches, larger-scale experiments, and more systematic ablation studies. At the same time, this also points to an important practical requirement for future work: strong empirical performance alone is not sufficient for real-world use; the models must also become efficient, robust, and deployable.

These limitations point naturally to several directions for future research. A first priority would be to extend the architectural comparison to multiclass classification and regression in a more systematic way. A second would be to investigate the effect of larger $n_{\max}$ and $d_{\max}$ during pretraining, as well as larger evaluation contexts at benchmark time. A third concerns prior design: the results indicate that this is one of the most important levers for downstream performance, and it would therefore be valuable to study more systematically which components of prior complexity are actually most beneficial. For instance, one could investigate priors that remain structurally simple but generate stronger class imbalance. Finally, both regression extensions deserve further development.

For the spline-based model, this could include more principled basis choices or adaptive knot placement. For the flow-based model, the most important next steps are likely to improve the robustness and calibration of the predictive distribution. More broadly, future work should also aim to make such models computationally efficient and practically deployable.

7 Conclusion

This thesis investigated TabPFN and developed several prototype models in order to better understand the design choices underlying the model type. A particular focus was placed on architectural decisions and on the choice of a suitable pretraining prior.

The results indicate that strong TabPFN performance depends on a match between prior complexity and model capacity. Both must be sufficiently expressive: empirical performance deteriorates when the pretraining prior is too simple, but increasing model complexity alone does not resolve this issue. In particular, a more expressive architecture cannot make full use of its capacity when it is pretrained on an overly simple prior. The findings therefore suggest that neither prior design nor model size should be considered in isolation; rather, both must be chosen in a compatible way.

In addition, the thesis introduced two new approaches to posterior predictive modeling in the regression setting. The spline-based approach appears to be a particularly practical extension of the existing framework, since it requires only limited changes while improving the continuity of the predictive distribution. The flow-based approach is more ambitious and opens a promising generative alternative, but it still requires further investigation.

A Appendix

A.1 Explorative analysis of the latent space

Since TabPFN-V2 (Hollmann et al., 2025) uses an alternating attention scheme, it becomes possible to analyze attention patterns across columns. Because the tasks are synthetic, the ground-truth feature weights are directly available and can therefore be related to the learned attention structure. This raises the question of whether columns of features that are more influential for the target also receive more attention.

To study this, we consider the simple-prior, since in this setting feature influence can be extracted in a particularly transparent way. Concretely, if w_j denotes the ground-truth coefficient associated with feature j , we define the relative feature importance as

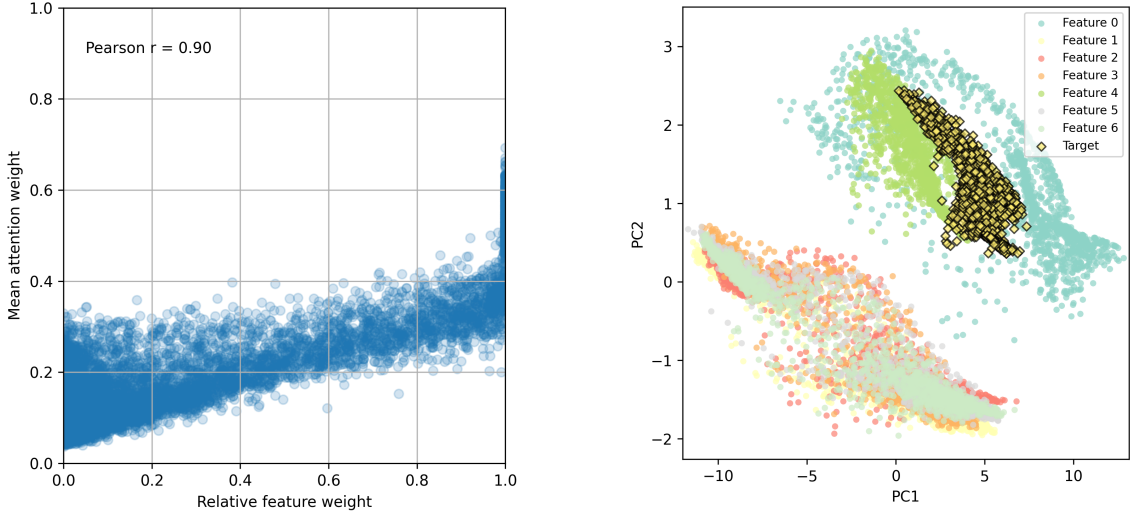
$$\text{importance_ratio}_j = \frac{|w_j|}{\sum_{k=1}^d |w_k|}, \quad j = 1, \dots, d.$$

This quantity serves as a simple proxy for how strongly each feature contributes to the target. We then compare this proxy to the attention mass assigned to the corresponding feature in the final layers of the model, averaged over samples and heads.

Figure 21a shows a clear positive relationship between this feature-importance proxy and the average attention weight assigned to a feature. In other words, features that play a stronger role in the data-generating mechanism also tend to receive more attention in the final layer of the model. This suggests that the learned attention patterns are not arbitrary, but may indeed be interpretable—at least in this simple setting.

It should be noted that some points have a relative feature weight of 1. These correspond to datasets containing only a single feature. Even in these cases, however, the average attention weight assigned to that feature is not 1 but rather centered around 0.5. The remaining attention mass is therefore allocated to the target token.

Figure 21b provides a complementary perspective by visualizing the hidden states of the final layer after dimensionality reduction with PCA. We use a similar illustration technique like Ye et al. (2025). Each point corresponds to one cell, i.e. to one (i, j) -combination, and is therefore associated with one token representation. Colors indicate different features, while the target tokens are shown in yellow with a black outline. The figure suggests that the model progressively enriches the internal representations across depth. Most notably, the point clouds of the more informative features are separated from those of the remaining, largely uninformative features. In the example shown in Figure 21b, the turquoise feature (feature 0) has an importance ratio of 0.13, while the light green feature (feature 4) has an importance ratio of 0.79—both relatively high compared to the remaining features. By contrast, the remaining features cluster in a similar region with comparable geometric structure. The two informative features, however, occupy different regions and exhibit distinct shapes, indicating that the model has learned to represent them differently. This provides additional evidence that the network identifies and internally distinguishes features that are more relevant for prediction.



(a) Relationship between feature importance and average attention weight in the final layer. (b) PCA projection of final-layer token representations, colored by feature identity.

Figure 21: Exploratory analysis of how TabPFN-V2 internally represents feature relevance under the simple prior.

A.2 Overview of TabPFN Versions

Table 3 provides an overview of the three TabPFN versions discussed in this thesis and summarizes their most relevant differences. Several broad trends can be observed. Most notably, successive versions were trained on increasingly large numbers of synthetic datasets and were designed to cover a broader range of dataset sizes. Interestingly, however, the number of trainable parameters decreased substantially from V1 to V2. This reduction is mainly due to the much smaller embedding dimension k , even though the overall architecture became more refined, most prominently through the introduction of alternating-attention.

For TabPFN-2.5, exact values are not publicly available for all entries, which is why some cells are marked with “—”. Nevertheless, since the model is presented as version 2.5 rather than as a completely new major version, it is reasonable to interpret it as an extension of the V2 design rather than as a fundamentally different architecture.

Attribute	TabPFN V1	TabPFN V2	TabPFN V2.5
n_{layers}	12	12	24 (classification), 18 (regression)
Attention scheme	1x2D attention	2x1D attention	2x1D attention
n_{datasets}	9.2M	130M	–
n_{max} (training)	1024	2048	–
d_{max} (training)	100	500	likely 2000
n_{max} (inference)	1,000	10,000	50,000
d_{max} (inference)	100	500	2,000
Train–test split	random	fixed 128 validation points	–
NaN handling	no	yes	yes
Categorical features	yes	yes	yes
k	512	192	–
m	1024	768	–
n_{heads}	4	6	–
Parameters	25.82M	7.24M	–
Tasks	class.	class. + regr.	class. + regr.

Table 3: Comparison of key design and training characteristics of TabPFN V1, V2, and V2.5. The information is compiled from Hollmann et al. (2023, 2025), Grinsztajn et al. (2025) and, where not explicitly reported in the corresponding publications, from the publicly available model implementations.

A.3 Additional details on the sweep for the simple-prior

This section provides additional details and findings for the sweep conducted under the simple prior.

The explored hyperparameter ranges are

$$\begin{aligned}
 k &\in [8, 128], & m &\in [8, 128], \\
 n_{\text{layers}} &\in \{1, 2, 3, 4, 6, 8, 12, 16\}, & n_{\text{heads}} &\in \{1, 2, 4, 8, 16\}, \\
 n_{\text{max}} &= 200, & d_{\text{max}} &= 20.
 \end{aligned}$$

Training was carried out with batch size 8, $n_{\text{datasets}} = 32,000$, learning rate 10^{-3} , and AdamW with weight decay 10^{-4} , without a learning-rate schedule. In total, the sweep comprises 500 trials. All runs were performed in parallel on several V100 GPUs.

Figure 22 shows the distribution of the difference in synthetic accuracy between V2 and V1, i.e. $\text{acc}_{\text{V2}} - \text{acc}_{\text{V1}}$. The vertical line at zero indicates equal performance. The histogram is clearly shifted to the right, suggesting that V2 achieves higher synthetic accuracy than V1 in the majority of configurations. At the same time, a smaller bump is visible on the left-hand side, corresponding to cases where V1 performs better. As will become clearer later, these cases are primarily associated with models that are too shallow.

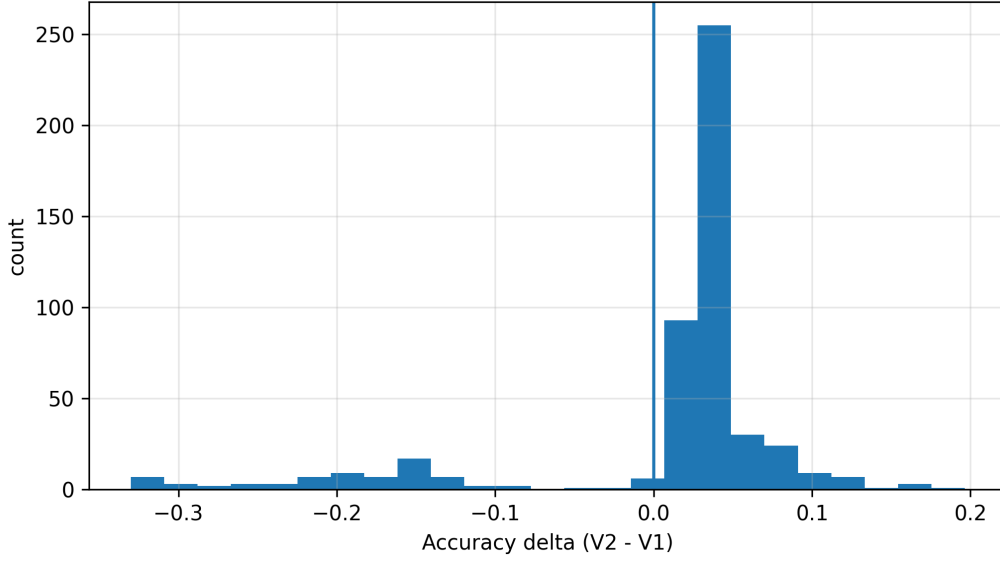


Figure 22: Distribution of the difference in synthetic accuracy between V2 and V1 across matched hyperparameter configurations under the simple prior. Positive values indicate better performance of V2.

Figure 23 shows the same general pattern as Figure 8 in Section 5.3.1, but in a more explicit paired form: V1 and V2 models with identical hyperparameter configurations are directly connected. The x-axis shows the synthetic accuracy of the respective model, while the y-axis gives the number of trainable parameters on a logarithmic scale.

As already noted in the main text, V2 lies quite consistently above V1, with the exception of a few shallow configurations in the low-performance regime. The connecting lines further show that V2 always has more trainable parameters than the corresponding V1 configuration, which is expected since its alternating-attention design introduces an additional attention mechanism. Moving toward the right-hand side of the plot, the discrepancy in parameter count between V1 and V2 becomes larger. This is mainly driven by configurations with large k , the primary determinant of parameter count. Because V2 includes an additional attention mechanism, the parameter gap between V1 and V2 grows especially strongly in this regime.

At the same time, the figure suggests that V1 performance no longer improves substantially once the parameter count becomes large. In other words, simply increasing the parameter count of V1 does not appear to yield major additional gains, which supports the view that the comparison between V1 and V2 is reasonably fair despite V2 consistently having more parameters. V1 would likely not benefit much from being scaled further. A similar saturation effect can also be observed for V2, although in that case many configurations already approach nearly perfect synthetic accuracy, suggesting that performance is close to the achievable optimum and that the remaining gap is largely due to aleatoric uncertainty.

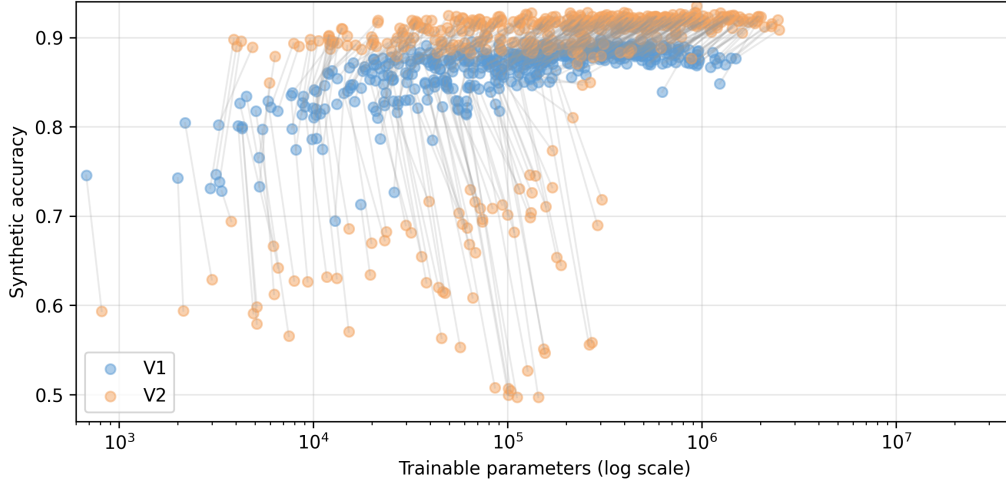


Figure 23: Synthetic accuracy versus number of trainable parameters for paired V1 and V2 models under identical hyperparameter configurations in the simple-prior sweep. Each line connects two models with the same configuration, making the systematic performance and parameter differences between V1 and V2 directly visible.

Figure 24 shows a faceted scatter plot of the four hyperparameters k , m , n_{layers} , and n_{heads} , with synthetic accuracy on the y-axis. Color indicates the fraction of evaluation datasets for which the model produced constant predictions, i.e. predicted only class 0 or only class 1 within one evaluation run. Overall, the picture is not particularly rich in structure. The main observations are that performance no longer improves substantially once k exceeds roughly 40, that m appears to have almost no effect, and that at least about four layers seem to be required, while additional depth matters little thereafter. For n_{heads} , no clear pattern is visible. The coloring further indicates that the model generally makes fairly differentiated predictions: brighter points would correspond to a higher fraction of constant predictions, but almost all points are very dark, suggesting that constant prediction behavior is rare for V1.

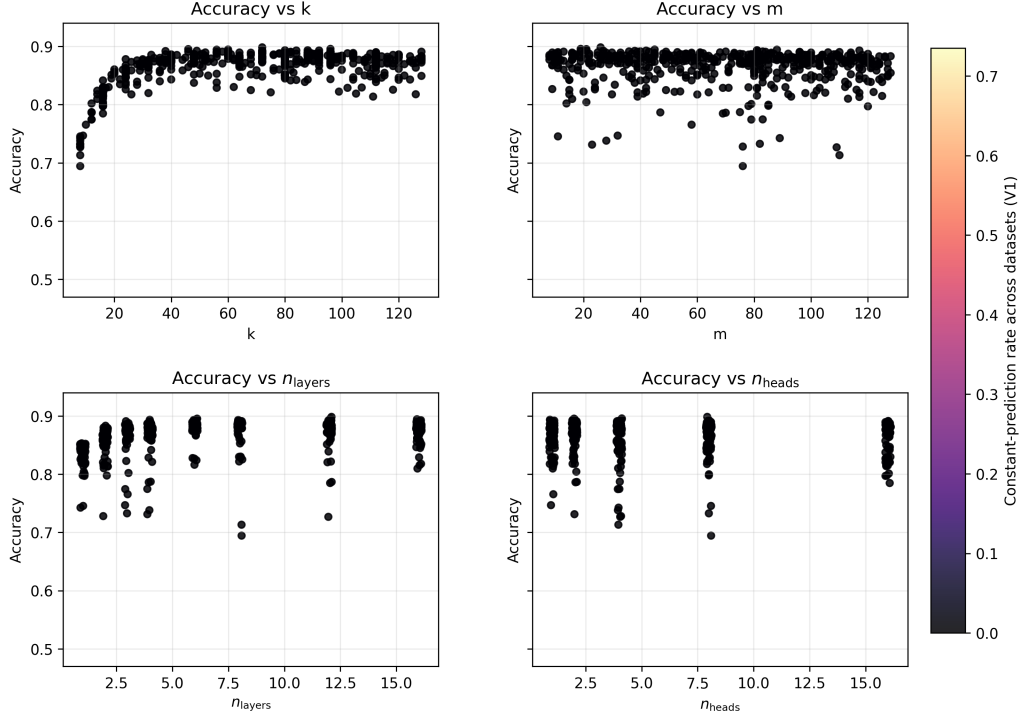


Figure 24: Synthetic accuracy of V1 under the simple prior as a function of each hyperparameter. Color indicates the fraction of evaluation datasets for which the model predicted a single class throughout an evaluation run.

Figure 25 shows the corresponding plot for V2 and reveals a more informative pattern. As in V1, increasing m appears to have little effect, and no strong structure is visible for this hyperparameter. For k , however, performance seems to saturate already around 20, and for very large k somewhat brighter points begin to appear, indicating a higher frequency of constant predictions on some datasets. The clearest pattern is observed for n_{layers} : shallow models suffer a substantial loss in performance, and this regime also appears to be associated with more frequent constant prediction behavior. The picture for n_{heads} is less pronounced, but there are some indications that very small numbers of heads are likewise associated with more constant predictions. Taken together, this suggests that V2 is more sensitive to hyperparameter choice than V1, in particular with respect to depth. One possible interpretation is that V2 requires a minimum level of depth in order to form sufficiently meaningful internal representations.

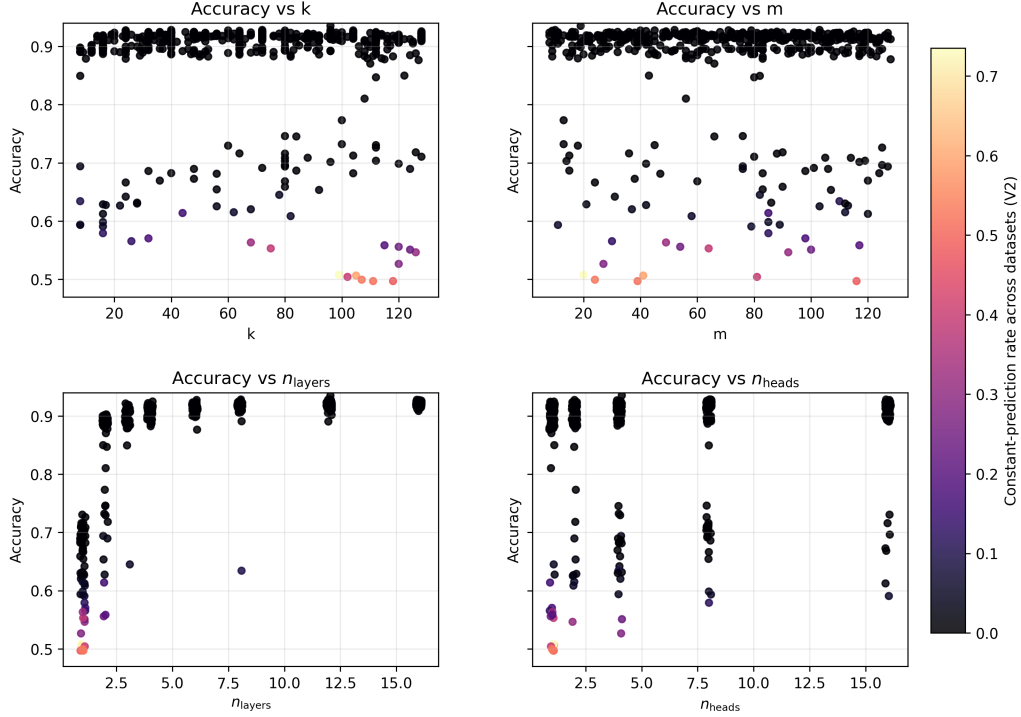


Figure 25: Synthetic accuracy of V2 under the simple prior as a function of each hyperparameter. Color indicates the fraction of evaluation datasets for which the model predicted a single class throughout an evaluation run.

Figure 26 shows a heatmap for the V1 architecture with n_{heads} on the x-axis and n_{layers} on the y-axis. Color indicates the median synthetic accuracy, where lighter colors correspond to better performance. Overall, the color range is fairly narrow, suggesting that most configurations perform reasonably well. The clearest exception is the row with only one layer, which is visibly worse than the others. The brightest region lies roughly in the center of the plot, indicating that the best-performing configurations tend to have moderate depth and a moderate number of heads. Toward the upper-right corner, performance becomes slightly weaker again. This supports the interpretation that V1 would likely not improve much further simply by making it deeper or by increasing the number of heads.

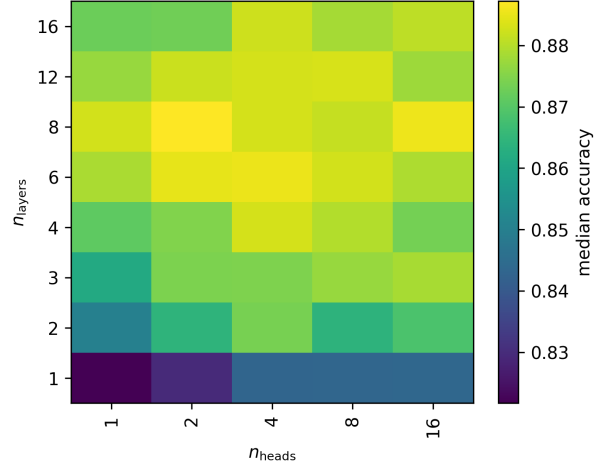


Figure 26: Median synthetic accuracy of V1 across combinations of depth and number of attention heads in the simple-prior sweep. Lighter colors indicate better performance.

Figure 27 shows the corresponding heatmap for V2 and reveals a clearly different pattern. In contrast to V1, the color scale spans a much wider range, from very strong performance down to values close to random prediction, i.e. around 50% accuracy. Most of the clearly underperforming models are concentrated in the row with only one layer, indicating that insufficient depth is the dominant failure mode. The number of attention heads appears to have a weaker effect, although performance improves slightly toward the right-hand side of the plot. Unlike the V1 heatmap, there is no visible deterioration in the upper-right region. This suggests that V2 may still benefit from additional depth or from a larger number of heads, in contrast to V1, where performance appears to saturate more clearly.

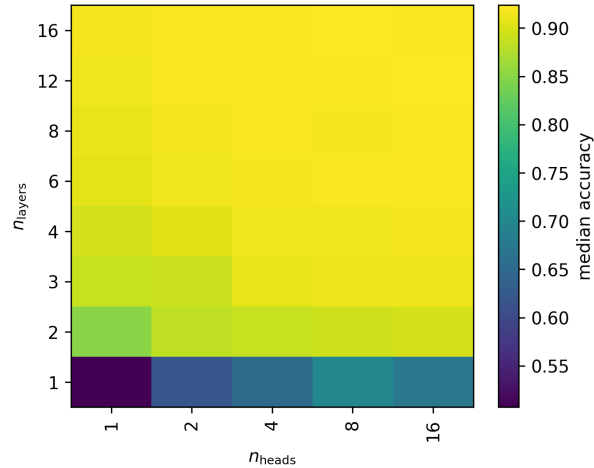


Figure 27: Median synthetic accuracy of V2 across combinations of depth and number of attention heads in the simple-prior sweep. Lighter colors indicate better performance.

A.4 Additional details on the sweep for the SCM prior

This section provides additional details for the sweep conducted under the SCM prior. Since this prior is considerably more complex than the simple prior, the training setup was adjusted accordingly. In particular, this sweep used linear warmup followed by cosine decay. The search space was also expanded, while training was carried out with $n_{\max} = 500$ and $d_{\max} = 20$. Due to the higher computational cost, the batch size had to be reduced to 1. The total number of synthetic training datasets was $n_{\text{datasets}} = 100,000$.

The explored hyperparameter ranges are

$$\begin{aligned} k &\in [8, 512], & m &\in [8, 512], \\ n_{\text{layers}} &\in \{1, 2, 3, 4, 6, 8, 12, 16\}, & n_{\text{heads}} &\in \{1, 2, 4, 8, 16\}, \\ n_{\max} &= 500, & d_{\max} &= 20. \end{aligned}$$

In total, the sweep comprises 281 trials. This smaller number of trials, relative to the simple-prior sweep, is mainly due to the substantially longer training times caused by the reduced batch size and the computationally demanding, CPU-heavy dataset generation.

Figure 28 shows the distribution of the difference in synthetic accuracy between V2 and V1, i.e. $\text{acc}_{V2} - \text{acc}_{V1}$, analogous to the histogram shown earlier for the simple-prior sweep. In contrast to the simple-prior case, the picture is more diffuse. The distribution is skewed, but a larger share of the mass appears to lie to the left of zero, indicating that V1 more often attains higher synthetic accuracy than V2 under the SCM-prior.

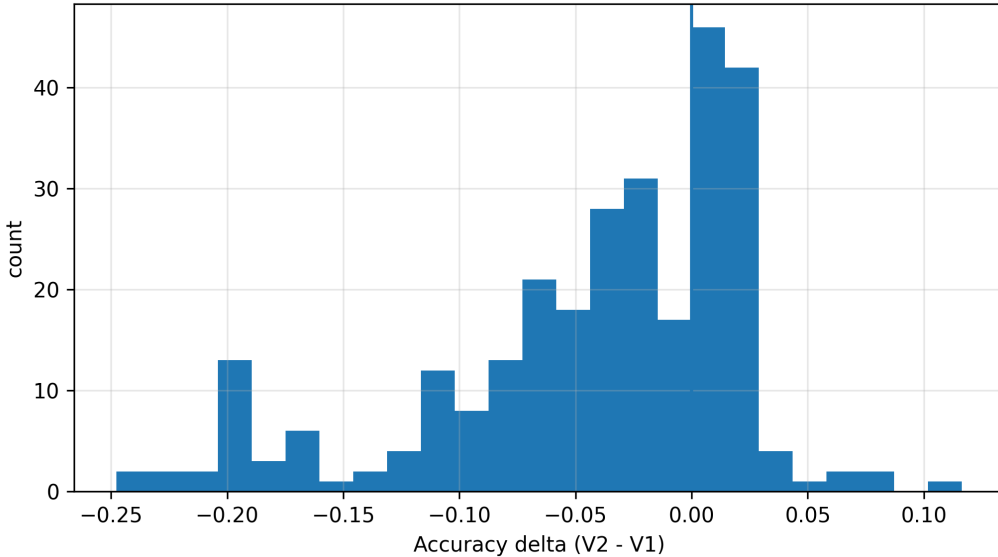


Figure 28: Distribution of the difference in synthetic accuracy between V2 and V1 across matched hyperparameter configurations under the SCM prior. Negative values indicate better performance of V1, positive values indicate better performance of V2.

Figure 29 again compares trainable parameter count and synthetic accuracy for paired V1 and V2 models with identical hyperparameter configurations. Relative to the simple-prior sweep, the point cloud is shifted downward and toward larger parameter counts,

reflecting both the greater difficulty of the SCM prior and the stronger emphasis on larger configurations in this sweep. The V1 models show comparatively stable performance, whereas V2 exhibits much greater variability in synthetic accuracy. At the same time, the best-performing configurations are still attained by V2. This reinforces the picture from the main text: V2 appears more sensitive to hyperparameter choice than V1, and this sensitivity seems even more pronounced under the SCM prior than under the simple prior.

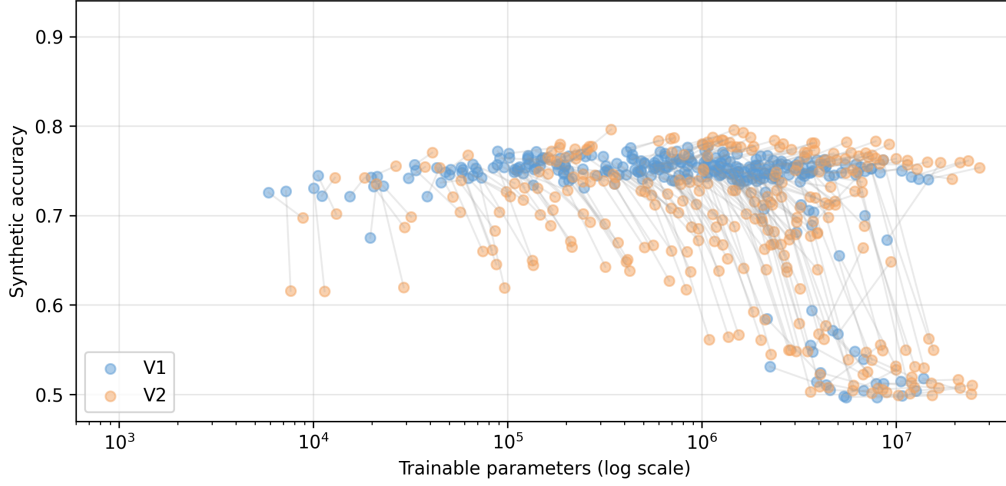


Figure 29: Synthetic accuracy versus number of trainable parameters for paired V1 and V2 models under identical hyperparameter configurations in the SCM-prior sweep. As in the simple-prior case, V2 consistently has more parameters than the corresponding V1 model, and this gap grows especially strongly on average for configurations with large k .

Figure 30 shows the same type of faceted scatter plot as before, now for V1 under the SCM prior. As in the simple-prior sweep, the relationship between most hyperparameters and performance is rather weak. The clearest pattern is that very large values of k , especially beyond roughly 300, appear to be associated with worse performance. In this regime, the model also seems more prone to degenerate behavior, i.e. constant or majority-class predictions. The remaining hyperparameters appear to have only a comparatively weak effect.

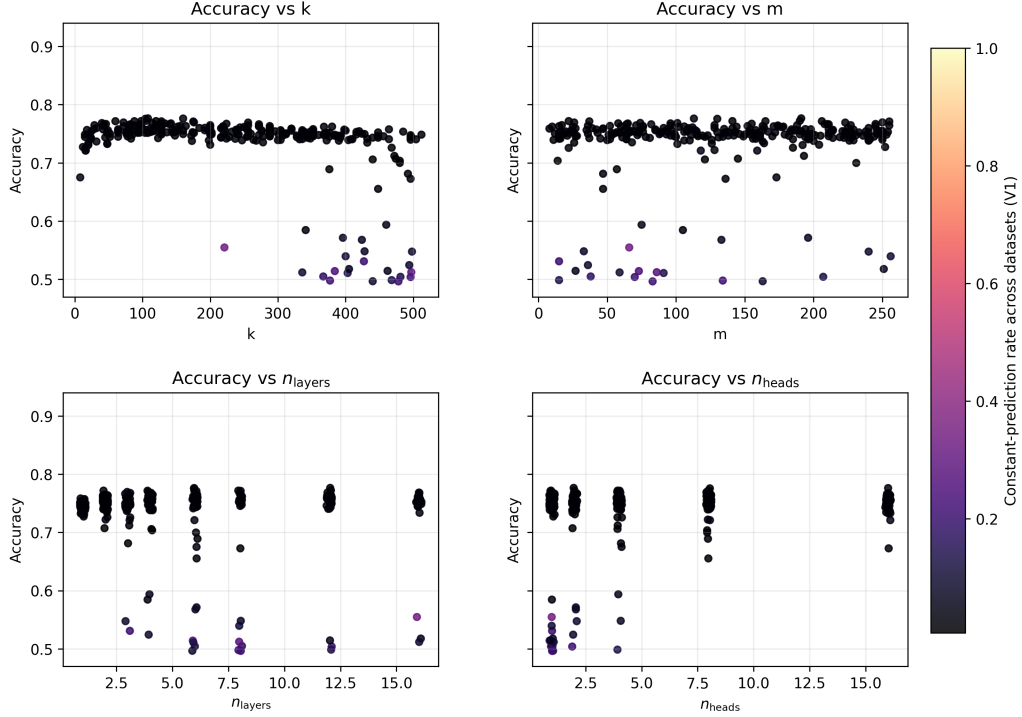


Figure 30: Synthetic accuracy of V1 under the SCM prior as a function of each hyperparameter. Color indicates the fraction of evaluation datasets for which the model predicted a single class throughout an evaluation run.

Figure 31 shows the corresponding plot for V2 and largely amplifies the patterns already visible for V1. The overall spread is larger, and very high values of k are now much more clearly associated with deteriorating performance. The number of layers also appears relevant: deeper models tend to perform better on average, although poor configurations still occur across this axis. As discussed in the main text, however, the main source of degradation in this sweep does not appear to be depth alone, but rather the effective per-head embedding dimension d_k . This suggests that, under the SCM prior, the way embedding capacity is distributed across heads matters more than the absolute model size alone.

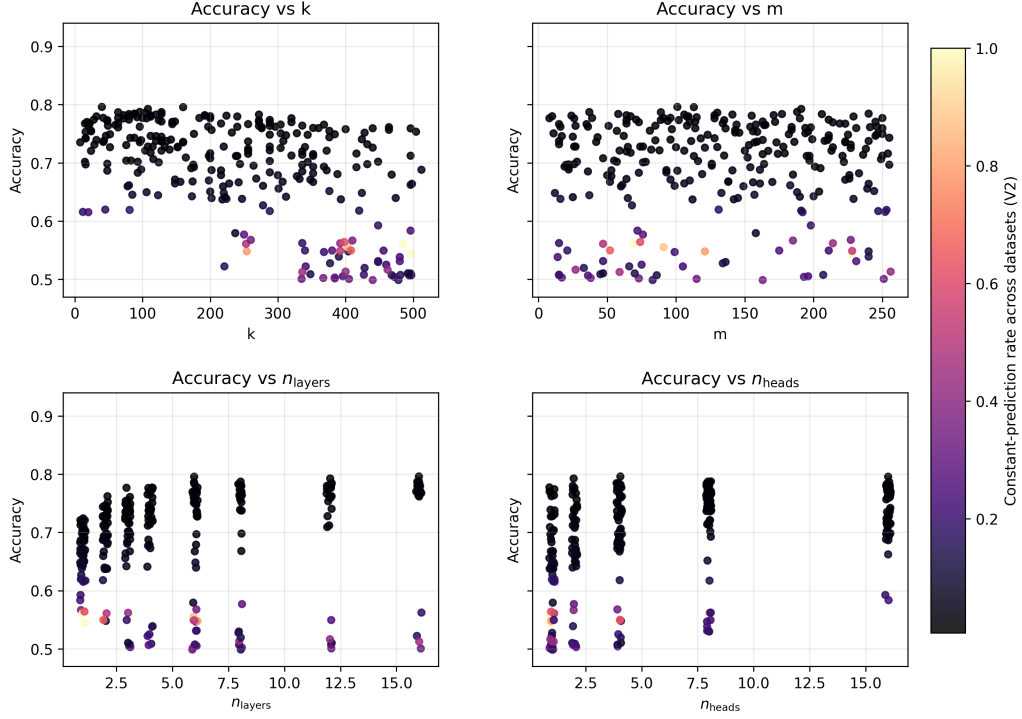


Figure 31: Synthetic accuracy of V2 under the SCM prior as a function of each hyperparameter. Color indicates the fraction of evaluation datasets for which the model predicted a single class throughout an evaluation run.

Figure 32 again shows a heatmap with n_{layers} on the y-axis and n_{heads} on the x-axis, now for V1 under the SCM prior. Compared to the simple-prior sweep, the pattern is more variable. Some configurations with many layers appear to perform particularly poorly, although this may partly reflect sampling noise, since the SCM sweep contains substantially fewer observations per cell than the simple-prior sweep. Apart from this, the overall picture remains fairly diffuse.

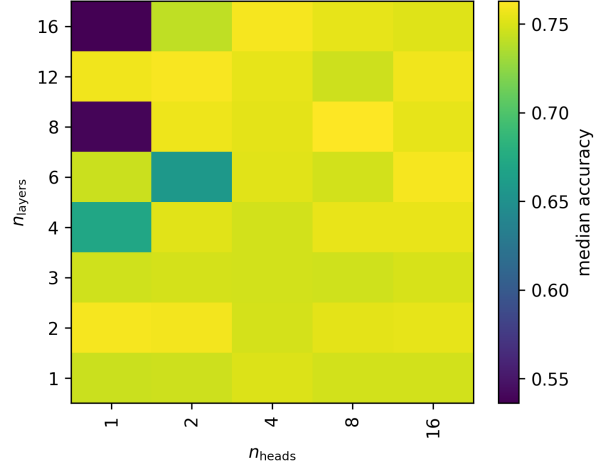


Figure 32: Median synthetic accuracy of V1 across combinations of depth and number of attention heads in the SCM-prior sweep. Lighter colors indicate better synthetic performance.

Figure 33 reveals a much more structured pattern for V2. In particular, the heatmap suggests an almost diagonal trend: strong configurations are concentrated in the upper-right region, indicating that good synthetic performance is obtained primarily when both n_{layers} and n_{heads} are sufficiently large. This is consistent with the interpretation developed in the main text, namely that V2 under the SCM prior benefits from both sufficient depth and a sufficiently fine allocation of embedding capacity across heads.

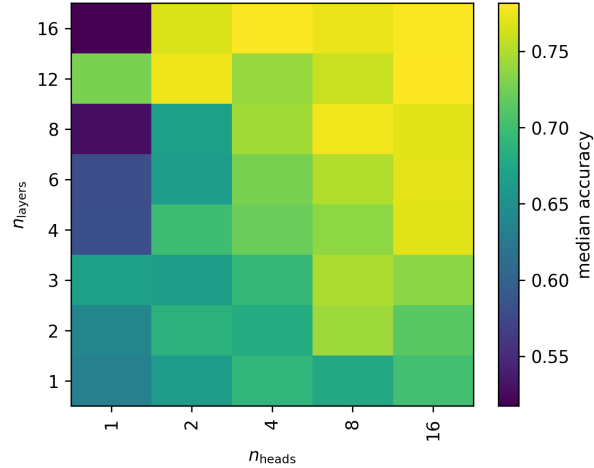


Figure 33: Median synthetic accuracy of V2 across combinations of depth and number of attention heads in the SCM-prior sweep. Lighter colors indicate better performance.

A.5 Performance over training at the dataset level

It is also informative to examine how generalization performance evolves over training for each benchmark dataset individually.

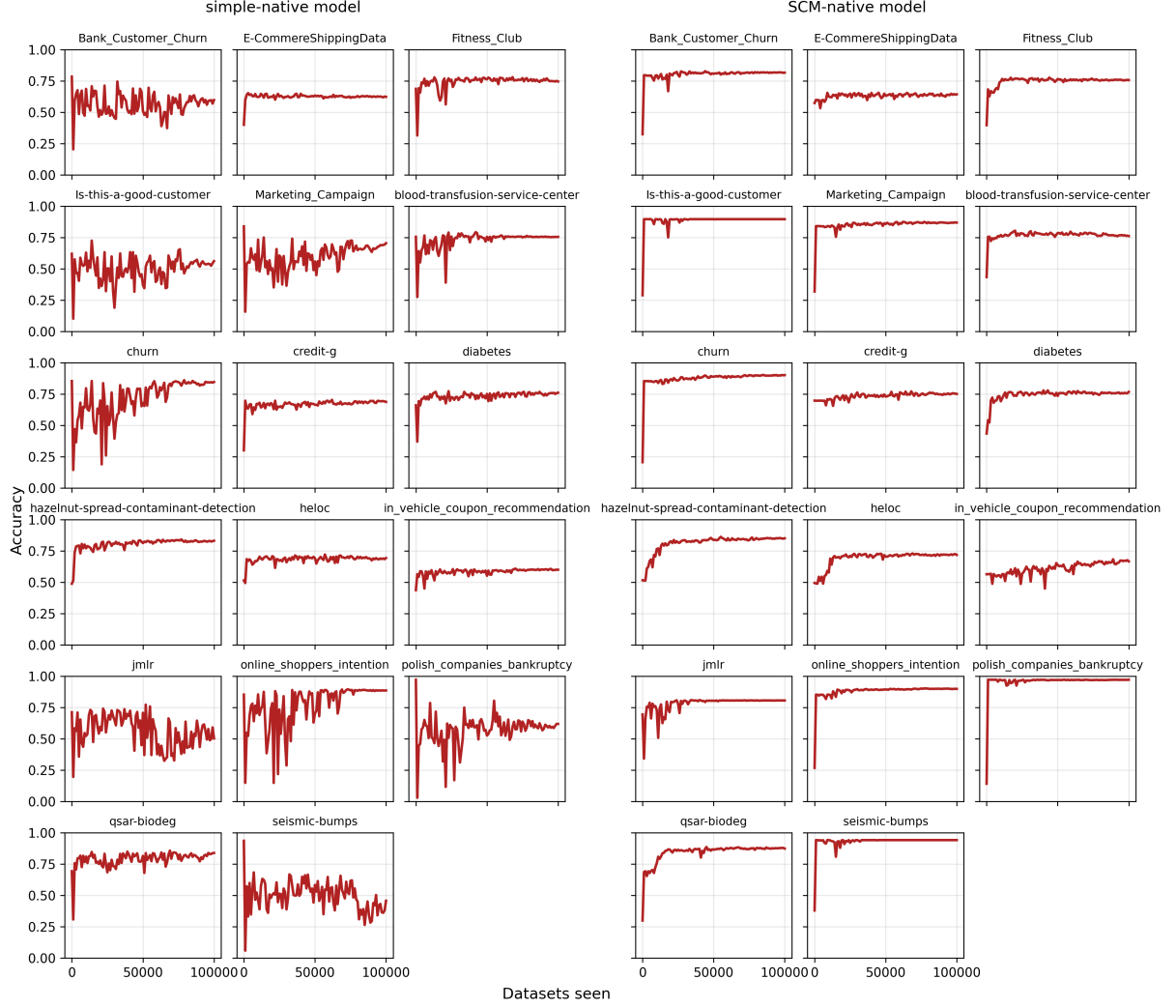


Figure 34: Dataset-wise development of real-data accuracy during training. Each facet corresponds to one benchmark dataset and shows how classification accuracy evolves over training checkpoints. The left block shows the simple-native model, whereas the right block shows the SCM-native model.

Figure 34 shows that the training trajectories of real-data accuracy are considerably less stable for the simple-native model (left block), especially in the early phase of training. By contrast, the SCM-native model (right block) exhibits a visibly smoother and more stable development across most datasets. In addition, it tends to reach a higher final accuracy level on the majority of datasets. A plausible interpretation is that pretraining on the more complex SCM prior leads to better threshold behavior, which is reflected more directly in the accuracy metric. Interestingly, datasets such as `seismic-bumps` and `jmlr` show a substantial late-stage drop in accuracy for the simple-native model. No similarly pronounced behavior is visible in the SCM-native case.

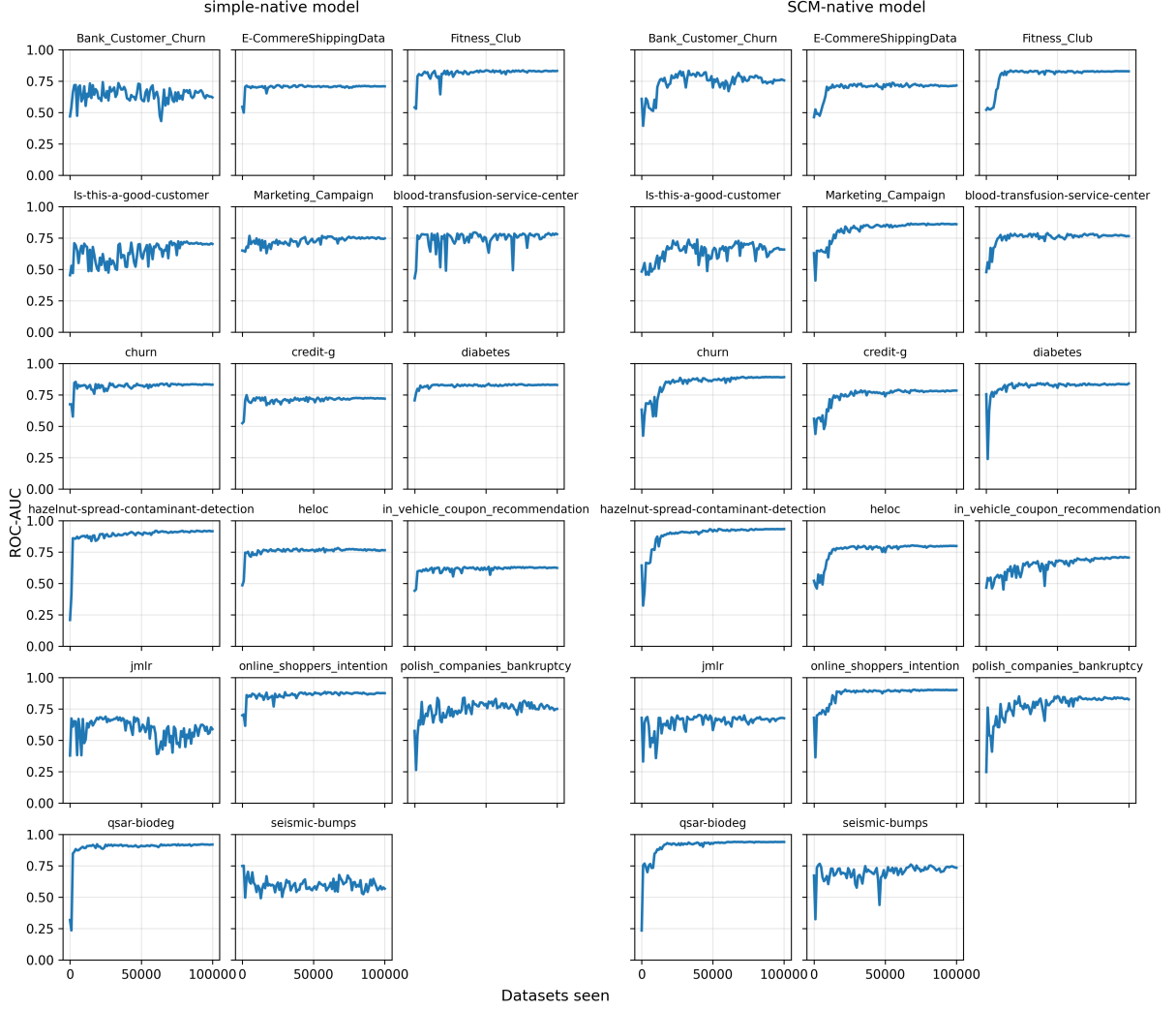


Figure 35: Dataset-wise development of real-data ROC-AUC during training. Each facet corresponds to one benchmark dataset and shows how ROC-AUC evolves over training checkpoints. The left block shows the simple-native model, whereas the right block shows the SCM-native model.

Figure 35 reveals a somewhat different pattern for ROC-AUC. Here, the convergence behavior is fairly similar across most datasets for the two models. Nevertheless, the SCM-native model often reaches a slightly higher final ROC-AUC level, indicating that the richer pretraining prior can also improve ranking performance, even if the difference is less pronounced than for accuracy. As in the accuracy plot, `jmlr` shows a mild decline in performance over training for the simple-native model.

When repeating the experiment with different random seeds, it also became apparent that the simple-native model can exhibit substantial variability across runs. In some cases, the resulting accuracy or ROC-AUC differed by as much as ten to twenty percentage points—at least for the datasets with an unstable trajectory. For all other datasets and especially when pretrained on the SCM-prior results were very stable over repeats.

A.6 Additional details on regression benchmarking

All regression models were trained with shared hyperparameters: `k=120`, `m=128`, `n_layers=12`, `n_heads=6`, `batch_size=4`, `n_datasets=1 500 000`, `base_lr=1e-4`, `min_lr=3e-6`, `accum_steps=4`, `task_type='SCM'`, `seed=0`.

Model-specific parameters and parameter counts are listed in Model-specific parameters and parameter counts are listed in Table 4. For clarity, naming conventions may differ slightly from those used in the implementation.

Model	Additional Parameters	Trainable Params
FlowPFN	<code>hidden_dim=100</code> , <code>y_emb_dim=128</code> , <code>t_fourier_dim=32</code>	1,977,213
Riemann	<code>n_bins=50</code>	1,778,786
Spline	<code>n_bins=50</code> , <code>degree=3</code>	1,779,509

Table 4: Model configurations for the regression benchmark.

A.7 Reproducibility and replicability

The reproducibility and replicability of the presented results differ across components of this work. The analysis notebooks (including all plots) for the classification and regression benchmarks and the hyperparameter analysis of the simple-prior sweep are fully reproducible: the corresponding notebooks, as well as the underlying datasets stored as Excel files, can be downloaded via the repository. Given access to the same pretrained models (which are also available for download), the reported benchmarking results can be reproduced to a large extent. Minor deviations were occasionally observed for the TabPFN baseline model, despite fixed random seeds; these variations appeared to be device-dependent and occurred only sporadically.

The proposed prototype models are deterministic at inference time when a fixed `torch.manual_seed` is set prior to the forward pass, as randomness arises only from feature perturbations in the encoder. However, the trained models themselves are only replicable, meaning that retraining typically yields models with comparable performance and convergence behavior, but not identical parameter values or outputs. In practice, we observed that for stable training setups, retrained models tend to converge to similar solutions. In more challenging regimes—such as when combining the simple-prior with over-parameterized models—the training process can become unstable, leading to high variability in the final parameter states. Additional discrepancies can arise because different versions of the models were trained at various stages of the project, during which the training procedures were iteratively refined (e.g., through the introduction of learning rate schedules). A similar limitation applies to the hyperparameter sweeps, which were executed in parallel across multiple devices with different random seeds. Consequently, while the overall findings are robust, exact reproduction of all training outcomes is not guaranteed.

B Electronic appendix

All code, data, pretrained models, and analysis notebooks are available at:

`https://github.com/alexanderulbrich/pfn\_prototype`

The repository includes instructions for reproducing all experiments and figures presented in this thesis. The experiments reported in this thesis correspond to commit 30c0952.

References

- Arik, S. Ö. and Pfister, T. (2021). Tabnet: Attentive interpretable tabular learning, *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35, pp. 6679–6687.
- Balazadeh, V., Kamkari, H., Thomas, V., Li, B., Ma, J., Cresswell, J. C. and Krishnan, R. G. (2025). Causalpfn: Amortized causal effect estimation via in-context learning, *arXiv preprint arXiv:2506.07918*.
- Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M. and Kasneci, G. (2022). Deep neural networks and tabular data: A survey, *IEEE transactions on neural networks and learning systems* **35**(6): 7499–7519.
- Breiman, L. (2001). Random forests, *Machine learning* **45**(1): 5–32.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. et al. (2020). Language models are few-shot learners, *Advances in neural information processing systems* **33**: 1877–1901.
- De Boor, C. (1986). B (asic)-spline basics., *Technical report*.
- Erickson, N., Purucker, L., Tschalzev, A., Holzmüller, D., Desai, P. M., Salinas, D. and Hutter, F. (2025). TabArena: A Living Benchmark for Machine Learning on Tabular Data. Version Number: 4.
URL: <https://arxiv.org/abs/2506.16791>
- Feuer, B., Schirrmeister, R. T., Cherepanova, V., Hegde, C., Hutter, F., Goldblum, M., Cohen, N. and White, C. (2024). Tunetables: Context optimization for scalable prior-data fitted networks, *Advances in Neural Information Processing Systems* **37**: 83430–83464.
- Gabriel, P. and Marco, C. (2019). Computational optimal transport with applications to data sciences, *Foundations and Trends® in Machine Learning* **11**(5-6): 355–607.
- Garnelo, M., Rosenbaum, D., Maddison, C., Ramalho, T., Saxton, D., Shanahan, M., Teh, Y. W., Rezende, D. and Eslami, S. A. (2018). Conditional neural processes, *International conference on machine learning*, PMLR, pp. 1704–1713.
- Gorishniy, Y., Rubachev, I., Khrulkov, V. and Babenko, A. (2021). Revisiting Deep Learning Models for Tabular Data, *Advances in Neural Information Processing Systems*, Vol. 34, Curran Associates, Inc., pp. 18932–18943.
URL: https://proceedings.neurips.cc/paper_files/paper/2021/hash/9d86d83f925f2149e9edb0ac3b492
Abstract.html
- Grinsztajn, L., Flöge, K., Key, O., Birkel, F., Jund, P., Roof, B., Jäger, B., Safaric, D., Alessi, S., Hayler, A., Manium, M., Yu, R., Jablonski, F., Hoo, S. B., Garg, A., Robertson, J., Bühler, M., Moroshan, V., Purucker, L., Cornu, C., Wehrhahn, L. C., Bonetto, A., Schölkopf, B., Gambhir, S., Hollmann, N. and Hutter, F. (2025). Tabpfn-2.5: Advancing the state of the art in tabular foundation models.
URL: <https://arxiv.org/abs/2511.08667>

- Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J. et al. (2020). Array programming with numpy, *nature* **585**(7825): 357–362.
- Hegselmann, S., Buendia, A., Lang, H., Agrawal, M., Jiang, X. and Sontag, D. (2023). Tabllm: Few-shot classification of tabular data with large language models, *International conference on artificial intelligence and statistics*, PMLR, pp. 5549–5581.
- Holderrieth, P. and Erives, E. (2025). An introduction to flow matching and diffusion models, *arXiv preprint arXiv:2506.02070*.
- Hollmann, N., Müller, S., Eggensperger, K. and Hutter, F. (2023). Tabpfn: A transformer that solves small tabular classification problems in a second, *International Conference on Learning Representations 2023*.
- Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirrmeister, R. T. and Hutter, F. (2025). Accurate predictions on small data with a tabular foundation model, *Nature*.
URL: <https://www.nature.com/articles/s41586-024-08328-6>
- Hoo, S. B., Müller, S., Salinas, D. and Hutter, F. (2024). The tabular foundation model tabpfn outperforms specialized time series forecasting models based on simple features, *NeurIPS workshop on time series in the age of large models*.
- Huang, X., Khetan, A., Cvitkovic, M. and Karnin, Z. (2020). Tabtransformer: Tabular data modeling using contextual embeddings, *arXiv preprint arXiv:2012.06678*.
- Hunter, J. D. (2007). Matplotlib: A 2d graphics environment, *Computing in science & engineering* **9**(3): 90–95.
- Islam, S., Elmekki, H., Elsebai, A., Bentahar, J., Drawel, N., Rjoub, G. and Pedrycz, W. (2024). A comprehensive survey on applications of transformers for deep learning tasks, *Expert Systems with Applications* **241**: 122666.
- Khan, S., Naseer, M., Hayat, M., Zamir, S. W., Khan, F. S. and Shah, M. (2022). Transformers in vision: A survey, *ACM computing surveys (CSUR)* **54**(10s): 1–41.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y. et al. (2023). Segment anything, *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4015–4026.
- Kramer, O. (2016). Scikit-learn, *Machine learning for evolution strategies*, Springer, pp. 45–53.
- Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M. and Le, M. (2022). Flow matching for generative modeling, *arXiv preprint arXiv:2210.02747*.
- Liu, S.-Y. and Ye, H.-J. (2025). Tabpfn unleashed: a scalable and effective solution to tabular classification problems. 2025, *URL* <https://arxiv.org/abs/2502.02527>.

- Lorch, L., Sussex, S., Rothfuss, J., Krause, A. and Schölkopf, B. (2022). Amortized inference for causal structure learning, *Advances in Neural Information Processing Systems* **35**: 13104–13118.
- Ma, J., Thomas, V., Hosseinzadeh, R., Labach, A., Cresswell, J. C., Golestan, K., Yu, G., Caterini, A. L. and Volkovs, M. (2025). Tabdpt: Scaling tabular foundation models on real data, *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Müller, S., Hollmann, N., Arango, S. P., Grabocka, J. and Hutter, F. (2021). Transformers can do bayesian inference, *arXiv preprint arXiv:2112.10510*.
- Nagler, T. (2023). Statistical Foundations of Prior-Data Fitted Networks, *Proceedings of the 40th International Conference on Machine Learning*, PMLR, pp. 25660–25676. ISSN: 2640-3498.
URL: <https://proceedings.mlr.press/v202/nagler23a.html>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. et al. (2019). Pytorch: An imperative style, high-performance deep learning library, *Advances in neural information processing systems* **32**.
- Peebles, W. and Xie, S. (2023). Scalable diffusion models with transformers, *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4195–4205.
- Pfefferle, A., Hog, J., Purucker, L. and Hutter, F. (2025). nanotabpfn: A lightweight and educational reimplementation of tabpfn, *arXiv preprint arXiv:2511.03634*.
- Popov, S., Morozov, S. and Babenko, A. (2019). Neural oblivious decision ensembles for deep learning on tabular data, *arXiv preprint arXiv:1909.06312*.
- Qiu, Z., Wang, Z., Zheng, B., Huang, Z., Wen, K., Yang, S., Men, R., Yu, L., Huang, F., Huang, S. et al. (2025). Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free, *arXiv preprint arXiv:2505.06708*.
- Qu, J., Holzmüller, D., Varoquaux, G. and Morvan, M. L. (2026). Tabiclv2: A better, faster, scalable, and open tabular foundation model, *arXiv preprint arXiv:2602.11139*.
- Qu, J., Holzmüller, D., Varoquaux, G. and Morvan, M. L. (2025). Tabicl: A tabular foundation model for in-context learning on large data, *arXiv preprint arXiv:2502.05564*.
- Schiff, D., Lindenbaum, O. and Efroni, Y. (2025). Gradient free deep reinforcement learning with tabpfn, *arXiv preprint arXiv:2509.11259*.
- Seabold, S., Perktold, J. et al. (2010). Statsmodels: econometric and statistical modeling with python., *scipy* **7**(1): 92–96.
- Sheather, S. J. (2004). Density estimation, *Statistical science* pp. 588–597.

- Somepalli, G., Goldblum, M., Schwarzschild, A., Bruss, C. B. and Goldstein, T. (2021). Saint: Improved neural networks for tabular data via row attention and contrastive pre-training, *arXiv preprint arXiv:2106.01342* .
- Thickstun, J. (2021). The transformer model in equations, *University of Washington: Seattle, WA, USA* .
- Torgo, L. and Gama, J. (1997). Regression using classification algorithms, *Intelligent Data Analysis* **1**(4): 275–292.
- Van Breugel, B. and Van Der Schaar, M. (2024). Why tabular foundation models should be a research priority, *arXiv preprint arXiv:2405.01147* .
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, u. and Polosukhin, I. (2017). Attention is All you Need, *Advances in Neural Information Processing Systems*, Vol. 30, Curran Associates, Inc.
URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J. et al. (2020). Scipy 1.0: fundamental algorithms for scientific computing in python, *Nature methods* **17**(3): 261–272.
- Voita, E., Talbot, D., Moiseev, F., Sennrich, R. and Titov, I. (2019). Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned, *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 5797–5808.
- Wang, Y., Jiang, B., Guo, Y., Gan, Q., Wipf, D., Huang, X. and Qiu, X. (2025). Prior-fitted networks scale to larger datasets when treated as weak learners, *arXiv preprint arXiv:2503.01256* .
- Waskom, M. L. (2021). Seaborn: statistical data visualization, *Journal of open source software* **6**(60): 3021.
- Wes McKinney (2010). Data Structures for Statistical Computing in Python, in Stéfan van der Walt and Jarrod Millman (eds), *Proceedings of the 9th Python in Science Conference*, pp. 56 – 61.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L. and Liu, T. (2020). On layer normalization in the transformer architecture, *International conference on machine learning*, PMLR, pp. 10524–10533.
- Ye, H.-J., Liu, S.-Y. and Chao, W.-L. (2025). A Closer Look at TabPFN v2: Understanding Its Strengths and Extending Its Capabilities. arXiv:2502.17361 [cs].
URL: <http://arxiv.org/abs/2502.17361>
- Yin, P., Neubig, G., Yih, W.-t. and Riedel, S. (2020). Tabert: Pretraining for joint understanding of textual and tabular data, *arXiv preprint arXiv:2005.08314* .

- Zeng, Y., Dinh, T., Kang, W. and Mueller, A. C. (2025). TabFlex: Scaling Tabular Learning to Millions with Linear Attention. arXiv:2506.05584 [cs].
URL: <http://arxiv.org/abs/2506.05584>
- Zhou, C., Li, Q., Li, C., Yu, J., Liu, Y., Wang, G., Zhang, K., Ji, C., Yan, Q., He, L. et al. (2025). A comprehensive survey on pretrained foundation models: A history from bert to chatgpt, *International Journal of Machine Learning and Cybernetics* **16**(12): 9851–9915.

Declaration of authorship

I hereby declare that the report submitted is my own unaided work. All direct or indirect sources used are acknowledged as references. I am aware that the Thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the report as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future Theses submitted. Further rights of reproduction and usage, however, are not granted here. This paper was not previously presented to another examination board and has not been published.

AI Tools

I hereby confirm that I wrote this thesis independently and take full responsibility for all of its contents, including all submitted code. I used ChatGPT-5, Microsoft Copilot, and Anthropic Claude Sonnet 4.6 as supporting tools in selected aspects of this work. All central research decisions, conceptual ideas, experimental design choices, interpretations, and conclusions are entirely my own. I am able to explain and justify all parts of the thesis and all adopted code independently.

General. AI tools were used as sparring partners for reviewing argument coherence, the logical flow of the thesis, and the consistency of mathematical notation and index conventions. Formulation suggestions were selectively reviewed and revised by me before adoption.

Mathematics. The concept of alternating-attention was developed in an iterative dialogue with ChatGPT-5, involving extensive back-and-forth in which many AI-generated suggestions were discarded; core conceptual contributions and decisions were made by me throughout. AI tools further assisted with index consistency across formulas and provided standard reference implementations (e.g., Cox-de Boor recursion), which I reviewed and integrated.

Programming. Conceptually central components—such as backbone architectures, encoders, and core task-generation logic—were implemented independently by me and subsequently verified in detail. AI tools were primarily used for computational refinement (e.g., batch-wise reformulations, debugging of shape mismatches, and assertions, buffering) and, for the flow-matching head, more extensively for standard building blocks such as sinusoidal embeddings and adaptive layer normalization. Training, evaluation, and hyperparameter sweep scripts were conceptually designed by me and refined with AI assistance. The same applies to synthetic data generation (simple-prior, SCM-prior) and bin construction, where core logic was implemented by me and auxiliary sub-functions were partly AI-generated and reviewed. Generic mathematical and numerical components—such as kernel density estimation, spline basis evaluations, and density sam-

pling—were largely drafted using AI tools (in line with standard reference implementations) and then reviewed and minimally adapted by me. Argument parsing, plotting routines, and L^AT_EX figure code were initially generated by AI tools based on my specifications and subsequently adapted. Generic utility scripts (e.g., `merge_sweeps.py`, `download_artifacts.py`, `paths.py`) were largely AI-generated and only minimally modified after careful review. Finally, AI-assisted code completion was used intermittently across the codebase for low-level implementation details and boilerplate, with all suggestions reviewed before integration.

Written Thesis. AI tools were used for linguistic and stylistic refinement throughout, including phrasing and readability suggestions, all of which were manually reviewed and revised. Intellectual content, structure, and argumentation are entirely my own.

Location, date

Name